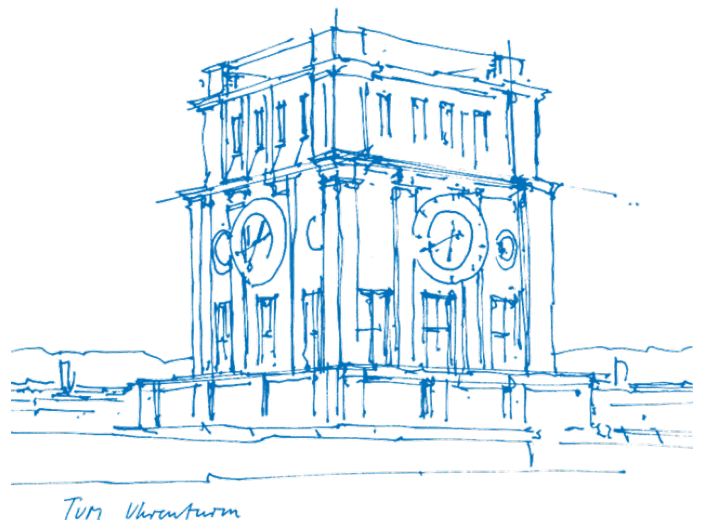


End-to-End Reinforcement Learning for Robust Teleoperation under Stochastic Delays

Kai-Ze, Deng



End-to-End Reinforcement Learning for Robust Teleoperation under Stochastic Delays

Kai-Ze, Deng

Thesis for the attainment of the academic degree

Master of Science (M.Sc.) in Robotics, Cognition and Intelligence

at the Department of Computer Engineering of the Technical University of Munich.

Examiner:

Prof. Dr. Alexander König

Supervisor:

Dr. Zewen Yang

Submitted:

Munich, 30.01.2026

I hereby declare that this thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references.

Munich, 30.01.2026

Kai-Ze, Deng

Abstract

Teleoperation systems enable human operators to control remote robots, supporting critical applications in remote surgery, distributed manufacturing, and space exploration. However, networked communication inevitably introduces stochastic, time-varying latencies in both the forward channel (operator commands to the follower robot) and the backward channel (follower robot feedback to the operator). These delays transform the control problem into a Partially Observable Decision Process, where the controller must act on outdated information. Tracking these discontinuous, irregularly-sampled reference points causes the follower robot to exhibit systematic lag and oscillatory behavior.

Recent approaches address this partial observability by combining neural network-based state prediction with reinforcement learning. However, these methods exhibit significant limitations under high-variance stochastic delays. Without explicit delay magnitude as input, the predictor cannot determine how far into the future it should predict, causing it to learn only an average prediction horizon that produces systematic errors when delays deviate from this average. Furthermore, single-step prediction architectures that update only upon packet arrival output identical predictions between arrivals, and when new packets finally arrive, the discontinuous jump in predicted states induces controller oscillations. Finally, two-stage approaches that use predicted states to tune PD gains inherit an inherent performance ceiling: the controller can only be as accurate as the upstream prediction, and the RL agent has no mechanism to compensate for prediction errors through direct control adaptation.

This thesis proposes an end-to-end reinforcement learning framework that jointly learns state prediction and control, replacing the conventional decoupled pipeline with a unified architecture optimized directly for task performance. The framework addresses the identified limitations through three design choices. First, the recurrent encoder receives explicit delay magnitude alongside delayed observations, enabling prediction horizons that adapt to varying delay lengths. Second, an autoregressive rollout mechanism feeds predictions back as inputs to maintain state predictions at the control frequency, preventing the stale outputs and resulting vibrations that occur when conventional predictors wait for new observations. Third, a Soft Actor-Critic policy maps latent embeddings directly to joint torques, eliminating the PD control structure and its sensitivity to discontinuous error signals. Joint optimization of encoder and policy enables delay-compensation strategies that generalize across varying delay conditions.

Experimental validation on a simulated Franka Panda manipulator under high-variance stochastic delays (90–290 ms) demonstrates that the proposed method maintains a mean tracking error of 0.02 m throughout operation. In contrast, SBSP achieves 0.04 m error and ASAC achieves 0.13 m error under the same delay conditions respectively, with both baselines exhibiting persistent oscillations.

Contents

Abstract	iv
1 Introduction	1
2 Preliminaries	4
2.1 Task Formulation	4
2.2 Time Discretization	4
2.3 Inverse Kinematics	4
2.4 Robot Dynamics	6
2.5 Long Short-Term Memory Networks	7
2.6 Reinforcement Learning	7
2.7 Soft Actor-Critic Algorithm	10
3 Related Work	12
3.1 Passivity-Based Control	12
3.2 Model-Based Predictive Control	12
3.3 Reinforcement Learning Control	12
3.3.1 Model-Free Approaches (State Augmentation)	13
3.3.2 Model-Based Approaches (Predictive Models)	13
4 Methodology	15
4.1 POMDP Formulation	15
4.1.1 State Space	15
4.1.2 Observation Space	16
4.1.3 Action Space	16
4.2 Reward Function	16
4.3 Termination Criteria	17
4.4 Network Architecture	18
4.4.1 LSTM-Based State Predictor	18
4.4.2 SAC Policy Network	19
4.5 Training Procedure	20
4.5.1 Prediction Loss	20
4.5.2 Behavioral Cloning	20
4.5.3 RL Fine-tuning	21
5 Experiments	23
5.1 Experimental Setup	23
5.2 Evaluation Metrics	24
5.3 Result Analysis	24
5.3.1 Experimental Protocol and Baseline Definitions	24
5.3.2 Performance on Fixed Trajectory	25
5.3.3 Generalization to Randomized Trajectories	27
5.3.4 Statistical Error Distribution	28

6	Results and Conclusion	30
6.1	Discussion	30
6.1.1	Architectural Efficiency and Inductive Bias	30
6.1.2	Direct Torque Control versus PD-Based Approaches	31
6.2	Limitations	32
6.3	Future Directions	33
6.4	Conclusion	33
A	Appendix	35
A.1	Notation	35
	Bibliography	38

1 Introduction

Teleoperation is a fundamental technology in modern robotics that enables human operators to control remote manipulators across geographical distances via networked communication. By bridging the gap between human expertise and remote environments, teleoperation facilitates expert intervention in scenarios where direct physical presence is impractical, hazardous, or impossible. This capability has driven transformative applications across multiple domains. In remote surgery, teleoperation allows surgeons to perform procedures on patients located in distant hospitals [1]. In distributed manufacturing, teleoperated systems enable skilled technicians to supervise and control production equipment across multiple facilities [2]. In space exploration and extreme terrestrial environments, teleoperation provides a means for ground-based operators to control robotic systems on orbital platforms or planetary surfaces [3, 4]. However, ensuring stability over networked channels remains a critical challenge [5], driving current research into shared control and autonomy [6].

Architecturally, a teleoperation system comprises two primary components: a leader device located at the operator site, which captures the human operator’s motion commands, and a follower manipulator at the remote site, which executes the corresponding movements in the distant environment (see Fig. 1.1). The leader and follower are connected through a networked communication channel that transmits command and feedback signals bidirectionally. However, the networked communication channel inevitably introduces fundamental challenges that degrade control performance. Specifically, the system is subject to three distinct sources of latency:

- **State Delay** ($\omega_s^t \in \mathbb{R}_{\geq 0}$): The network latency in transmitting the leader’s state signal to the follower site. At any time t , the controller at the follower site receives commands that were issued at time $t - \omega_s^t$.
- **Action Delay** ($\omega_a \in \mathbb{R}_{\geq 0}$): The local system latency between the controller issuing a torque command and the actuators executing it. A command issued at time t effectively acts upon the physical system at time $t + \omega_a$.
- **Observation Delay** ($\omega_o \in \mathbb{R}_{\geq 0}$): The local sensor latency in transmitting feedback from the robot’s physical state back to the controller. Consequently, the controller observes the state corresponding to time $t - \omega_o$.

Crucially, these delays differ in nature. Action and observation delays are local system latencies; they are generally constant and deterministic, as the controller resides at the follower site and communicates with hardware via a high-speed local bus. In contrast, the state delay is induced by the network forward channel and is inherently stochastic. Due to network congestion, routing variability, and packet retransmission, operator commands arrive at irregular intervals, resulting in a discontinuous, non-uniformly sampled command sequence.

In this work, we focus on the controller’s partial observability problem induced by stochastic state delay and do not consider a feedback path from the controller to the operator. This scope is appropriate for applications where the operator provides high-level trajectory commands and the RL policy autonomously handles delay compensation at the follower site.

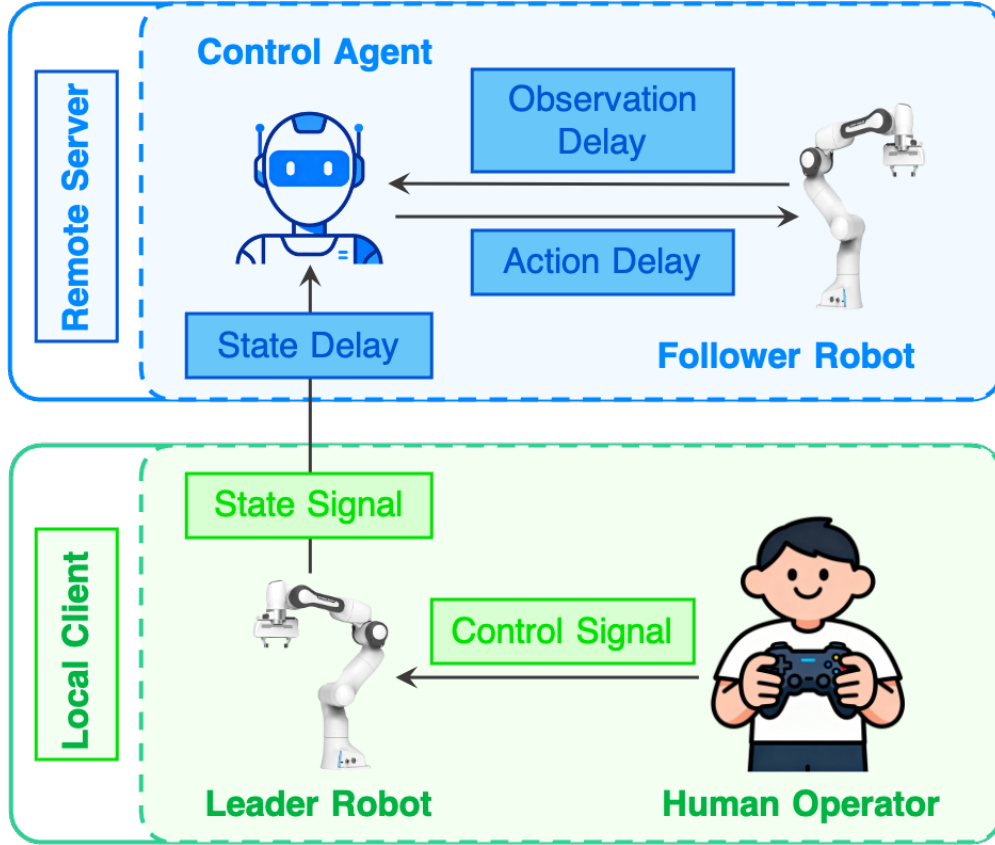


Figure 1.1 Teleoperation architecture with stochastic communication delays. The time-varying state delay ω_s^t is induced by client-server transmission latency; the action delay ω_a and observation delay ω_o arise from local communication within the agent-follower control loop and remain constant.

The stochastic nature of state delay poses fundamental challenges for conventional PD controllers, which exhibit high-frequency oscillations when error signals fluctuate due to variable latency. These limitations have motivated various delay compensation approaches, including passivity-based control, model-based prediction, and learning-based methods (A detailed review of these approaches is provided in Chapter 3). Among these, reinforcement learning has emerged as a promising direction due to its model-free nature and ability to learn adaptive policies. However, standard RL algorithms are designed for Markov Decision Processes, which assume the agent has access to the complete current state at each decision step. Under stochastic state delay, the agent receives only outdated leader commands and delayed follower observations, violating the Markov assumption and transforming the problem into a Partially Observable Markov Decision Process (POMDP).

To address this partial observability, recent work has combined model-based state prediction with reinforcement learning [7]. However, these two-stage approaches—predicting states then tuning PD gains—exhibit fundamental limitations under high-variance delays: they lack explicit delay conditioning, produce discontinuous estimates between packet arrivals, and propagate prediction errors through rigid linear control structures. These limitations motivate an end-to-end approach that directly maps delayed observations to control torques.

This thesis proposes an end-to-end reinforcement learning framework that addresses the identified limitations by jointly learning state prediction and direct torque control. The primary contributions are:

1. **Delay-adaptive autoregressive state prediction.** An LSTM-based encoder receives explicit delay magnitude alongside delayed observations, enabling prediction horizons that adapt to varying delay lengths. An autoregressive rollout mechanism feeds predictions back as inputs to maintain state estimates at control frequency, preventing stale outputs when no new observations arrive.

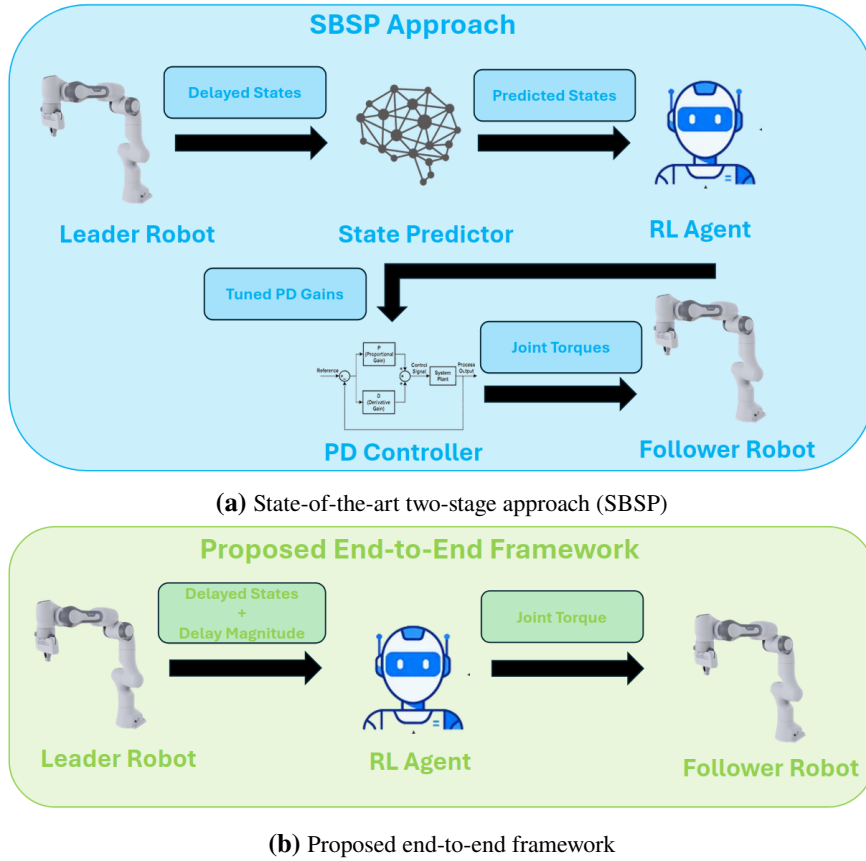


Figure 1.2 Architecture comparison. (a) SOTA method separates state prediction from PD gain tuning, with the RL agent outputting controller gains rather than direct torques. (b) The proposed approach integrates state prediction and control into a unified policy that directly outputs joint torques.

2. **Direct torque control via reinforcement learning.** A Soft Actor-Critic policy maps latent state embeddings directly to joint torques, bypassing the PD control structure and eliminating sensitivity to discontinuous error signals.
3. **Experimental validation under diverse delay conditions.** We validate the framework on a simulated Franka Panda manipulator across three delay scenarios: low delay with low variance, high delay with low variance, and high delay with high variance.

2 Preliminaries

This chapter establishes the technical foundations for the proposed framework.

2.1 Task Formulation

The objective is to achieve high-fidelity motion synchronization between the leader and follower robots. Let $\mathbf{q}_l^t \in \mathbb{R}^n$ and $\mathbf{q}_f^t \in \mathbb{R}^n$ denote the joint configurations of the leader and follower robots at time t , respectively. Let $\mathbf{x}_l^t \in \mathbb{R}^m$ and $\mathbf{x}_f^t \in \mathbb{R}^m$ denote the end-effector positions of the leader and follower robots in task space, respectively.

The human operator controls the leader robot in task space by specifying end-effector positions $\mathbf{x}_l^t$. These task-space commands are converted to joint-space configurations via inverse kinematics:

$$\mathbf{q}_l^t = \mathbf{f}^{-1}(\mathbf{x}_l^t), \quad (2.1)$$

where $\mathbf{f}^{-1}(\cdot) : \mathbb{R}^m \rightarrow \mathbb{R}^n$ is the inverse kinematics mapping (Section 2.3). The resulting joint-space command $\mathbf{q}_l^t$ is transmitted to the follower over the network.

The joint tracking error is defined as:

$$\mathbf{e}^t = \mathbf{q}_l^t - \mathbf{q}_f^t. \quad (2.2)$$

The goal is to design a control policy π that minimizes the tracking error $\|\mathbf{e}^t\|$ under stochastic communication delays.

2.2 Time Discretization

While the system is defined in continuous time t , the control loop operates in discrete time steps k . The controller runs at a fixed frequency f_c (Hz), with time step duration $\Delta t = 1/f_c$. The continuous time corresponding to step k is $t = k\Delta t$.

The stochastic state delay ω_s^k , measured in milliseconds, corresponds to a discrete delay of $d_s^k \in \mathbb{Z}_{\geq 0}$ time steps:

$$d_s^k = \left\lfloor \frac{\omega_s^k}{\Delta t \cdot 10^3} \right\rfloor. \quad (2.3)$$

Similarly, the constant observation delay ω_o and action delay ω_a , also measured in milliseconds, correspond to fixed discrete delays $d_o, d_a \in \mathbb{Z}_{\geq 0}$:

$$d_o = \left\lfloor \frac{\omega_o}{\Delta t \cdot 10^3} \right\rfloor, \quad d_a = \left\lfloor \frac{\omega_a}{\Delta t \cdot 10^3} \right\rfloor. \quad (2.4)$$

2.3 Inverse Kinematics

Inverse kinematics (IK) computes the joint configuration $\mathbf{q} \in \mathbb{R}^n$ required to place the robot's end-effector at a desired position $\mathbf{x}_d \in \mathbb{R}^m$ in task space. Given the forward kinematics mapping $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$, the IK problem seeks:

$$\mathbf{q} = \mathbf{f}^{-1}(\mathbf{x}_d). \quad (2.5)$$

Jacobian-Based Iterative Solution

For general manipulators, iterative numerical methods are required for solving IK equation. The Jacobian-based approach is widely used: it linearizes the nonlinear IK problem around the current configuration and iteratively refines the joint angles until the end-effector reaches the desired position.

The Jacobian matrix $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{m \times n}$ relates differential changes in joint space to differential changes in task space:

$$\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}. \quad (2.6)$$

For small displacements, this relationship can be approximated as:

$$\Delta \mathbf{x} \approx \mathbf{J}(\mathbf{q})\Delta \mathbf{q}, \quad (2.7)$$

where $\Delta \mathbf{x} \in \mathbb{R}^m$ is a small end-effector displacement and $\Delta \mathbf{q} \in \mathbb{R}^n$ is the corresponding joint displacement.

Given the current end-effector position $\mathbf{x}$ and desired position $\mathbf{x}_d$, the task-space error is defined as $\mathbf{e}_x = \mathbf{x}_d - \mathbf{x}$. The joint correction $\Delta \mathbf{q}$ that reduces this error is obtained by solving:

$$\mathbf{J}(\mathbf{q})\Delta \mathbf{q} = \mathbf{e}_x. \quad (2.8)$$

For robots with more joints than task-space dimensions ($n > m$), such as the 7-DoF Franka Panda used in this work, the system is underdetermined and admits infinitely many solutions. The minimum-norm solution is given by the Moore-Penrose pseudoinverse:

$$\Delta \mathbf{q} = \mathbf{J}^\dagger(\mathbf{q})\mathbf{e}_x, \quad (2.9)$$

where $\mathbf{J}^\dagger \in \mathbb{R}^{n \times m}$ denotes the pseudoinverse of $\mathbf{J}(\mathbf{q})$.

Since the linear approximation in Equation (2.7) is only valid for small displacements, the IK solution is computed iteratively. At each iteration i , the joint configuration is updated as:

$$\mathbf{q}^{(i+1)} = \mathbf{q}^{(i)} + \alpha \mathbf{J}^\dagger(\mathbf{q}^{(i)})\mathbf{e}_x^{(i)}, \quad (2.10)$$

where $\alpha \in (0, 1]$ is the step size and $\mathbf{e}_x^{(i)} = \mathbf{x}_d - \mathbf{x}^{(i)}$ is the task-space error at iteration i , with $\mathbf{x}^{(i)}$ denoting the end-effector position corresponding to $\mathbf{q}^{(i)}$. The iteration terminates when the position error falls below a specified tolerance ϵ :

$$\|\mathbf{e}_x^{(i)}\| < \epsilon. \quad (2.11)$$

Inertia-Weighted Damped Least Squares

The pseudoinverse solution in Equation (2.9) has two practical limitations. First, when the robot approaches certain configurations where the end-effector loses mobility in one or more directions (known as singular configurations), the pseudoinverse becomes numerically unstable and produces excessively large joint velocities. Second, for robots with more joints than required for the task, the pseudoinverse selects one solution among infinitely many without considering which solution is most efficient for the robot to execute.

To address these issues, we employ the damped least squares method with inertia weighting:

$$\mathbf{J}^\dagger = \mathbf{M}^{-1}(\mathbf{q})\mathbf{J}^T \left(\mathbf{J}\mathbf{M}^{-1}(\mathbf{q})\mathbf{J}^T + \lambda^2 \mathbf{I} \right)^{-1}, \quad (2.12)$$

where $\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{n \times n}$ is the joint-space inertia matrix from Equation (2.15), $\mathbf{I}$ is the identity matrix, and $\lambda > 0$ is a damping factor.

The damping term $\lambda^2 \mathbf{I}$ prevents numerical instability near singular configurations by bounding the magnitude of the solution. The inertia weighting $\mathbf{M}^{-1}(\mathbf{q})$ biases the solution toward motions that require less effort: joints with higher inertia contribute less to the movement, resulting in more energy-efficient trajectories.

Null-Space Projection

For robots with more joints than task-space dimensions, multiple joint configurations can achieve the same end-effector position. This means certain joint motions produce no end-effector displacement at all. These motions span what is mathematically termed the null space of the Jacobian.

This additional freedom can be exploited to achieve a secondary objective while the robot performs its primary tracking task. The null-space projector is defined as:

$$N = I - J^\dagger J. \quad (2.13)$$

Any joint motion projected through N is guaranteed not to affect the end-effector position.

In this work, the secondary objective drives the manipulator toward a comfortable rest configuration $\mathbf{q}_{\text{rest}}$, which helps avoid joint limits and maintains favorable positioning for future movements. The complete joint correction combines the primary tracking task with the null-space term:

$$\Delta \mathbf{q} = J^\dagger \mathbf{e}_x + N (k_{\text{null}} (\mathbf{q}_{\text{rest}} - \mathbf{q})), \quad (2.14)$$

where $k_{\text{null}} > 0$ is a gain that controls the strength of the secondary objective. The first term achieves the desired end-effector motion, while the second term adjusts the robot's posture without interfering with end-effector tracking.

2.4 Robot Dynamics

This work considers a homogeneous teleoperation system where the leader and follower robots share the same embodiment. Each robot is modeled as a rigid-body mechanical system with n degrees of freedom. The equation of motion for the follower robot is:

$$\mathbf{M}(\mathbf{q}_f) \ddot{\mathbf{q}}_f + \mathbf{C}(\mathbf{q}_f, \dot{\mathbf{q}}_f) \dot{\mathbf{q}}_f + \mathbf{D} \dot{\mathbf{q}}_f + \mathbf{g}(\mathbf{q}_f) = \boldsymbol{\tau}, \quad (2.15)$$

where $\mathbf{q}_f, \dot{\mathbf{q}}_f, \ddot{\mathbf{q}}_f \in \mathbb{R}^n$ are the joint position, velocity, and acceleration vectors, $\mathbf{M}(\mathbf{q}_f) \in \mathbb{R}^{n \times n}$ is the inertia matrix, $\mathbf{C}(\mathbf{q}_f, \dot{\mathbf{q}}_f) \in \mathbb{R}^{n \times n}$ is the Coriolis and centrifugal matrix, $\mathbf{D} \in \mathbb{R}^{n \times n}$ is the damping matrix, $\mathbf{g}(\mathbf{q}_f) \in \mathbb{R}^n$ is the gravitational torque vector, and $\boldsymbol{\tau} \in \mathbb{R}^n$ is the control torque vector.

Forward Dynamics

Forward dynamics computes the joint accelerations resulting from applied torques. In torque-controlled systems, the controller outputs a torque command $\boldsymbol{\tau}^k$ at each time step k . The system dynamics determine the instantaneous joint acceleration $\ddot{\mathbf{q}}_f^k$ according to:

$$\ddot{\mathbf{q}}_f^k = \mathbf{M}^{-1}(\mathbf{q}_f^k) \left[\boldsymbol{\tau}^k - \mathbf{C}(\mathbf{q}_f^k, \dot{\mathbf{q}}_f^k) \dot{\mathbf{q}}_f^k - \mathbf{D} \dot{\mathbf{q}}_f^k - \mathbf{g}(\mathbf{q}_f^k) \right]. \quad (2.16)$$

This acceleration is subsequently integrated to compute the state at the next time step, $(\mathbf{q}_f^{k+1}, \dot{\mathbf{q}}_f^{k+1})$. In this work, the RL policy directly outputs joint torques for controlling the follower robot.

Inverse Dynamics

Inverse dynamics computes the torque required to achieve a desired joint acceleration. Given a target acceleration $\ddot{\mathbf{q}}_d^k$ and the current state $(\mathbf{q}_f^k, \dot{\mathbf{q}}_f^k)$ at time step k , the required torque is obtained directly from Equation (2.15):

$$\boldsymbol{\tau}^k = \mathbf{M}(\mathbf{q}_f^k) \ddot{\mathbf{q}}_d^k + \mathbf{C}(\mathbf{q}_f^k, \dot{\mathbf{q}}_f^k) \dot{\mathbf{q}}_f^k + \mathbf{D} \dot{\mathbf{q}}_f^k + \mathbf{g}(\mathbf{q}_f^k). \quad (2.17)$$

2.5 Long Short-Term Memory Networks

Long Short-Term Memory (LSTM) networks [8] are recurrent architectures designed to learn long-range dependencies in sequential data. Unlike standard recurrent networks that suffer from vanishing gradients over long sequences, LSTMs maintain information through a gated cell state that enables stable gradient flow during training.

An LSTM maintains two internal states: a cell state $\mathbf{c}^k \in \mathbb{R}^{d_h}$ for long-term memory and a hidden state $\mathbf{h}^k \in \mathbb{R}^{d_h}$ as the output at each time step. Information flow is controlled by three gates. At time step k , given input $\mathbf{u}^k$ and previous states $(\mathbf{h}^{k-1}, \mathbf{c}^{k-1})$, the LSTM computes:

$$\mathbf{f}^k = \sigma(\mathbf{W}_f \mathbf{u}^k + \mathbf{U}_f \mathbf{h}^{k-1} + \mathbf{b}_f), \quad (2.18)$$

$$\mathbf{i}^k = \sigma(\mathbf{W}_i \mathbf{u}^k + \mathbf{U}_i \mathbf{h}^{k-1} + \mathbf{b}_i), \quad (2.19)$$

$$\mathbf{o}^k = \sigma(\mathbf{W}_o \mathbf{u}^k + \mathbf{U}_o \mathbf{h}^{k-1} + \mathbf{b}_o), \quad (2.20)$$

where $\sigma(\cdot)$ is the sigmoid function. The forget gate $\mathbf{f}^k$ controls what information to discard, the input gate $\mathbf{i}^k$ controls what new information to store, and the output gate $\mathbf{o}^k$ determines what to output.

The cell and hidden states are updated as:

$$\tilde{\mathbf{c}}^k = \tanh(\mathbf{W}_c \mathbf{u}^k + \mathbf{U}_c \mathbf{h}^{k-1} + \mathbf{b}_c), \quad (2.21)$$

$$\mathbf{c}^k = \mathbf{f}^k \odot \mathbf{c}^{k-1} + \mathbf{i}^k \odot \tilde{\mathbf{c}}^k, \quad (2.22)$$

$$\mathbf{h}^k = \mathbf{o}^k \odot \tanh(\mathbf{c}^k), \quad (2.23)$$

where $\odot$ denotes element-wise multiplication. The additive cell state update allows information to flow across many time steps when the forget gate is close to one, enabling learning over long sequences.

This architecture is particularly well-suited for teleoperation under stochastic delays, where the effective observation history length d_s^k varies dynamically. The gating mechanism allows the network to selectively retain relevant trajectory dynamics in the cell state $\mathbf{c}^k$ while ignoring irrelevant or stale data, providing a robust internal representation even when input sequences are irregular or interrupted.

2.6 Reinforcement Learning

Reinforcement learning (RL) is a framework for sequential decision-making where an agent learns to maximize cumulative rewards through interaction with an environment. In this work, RL serves as an alternative to conventional controllers for the follower robot, learning a control policy that tracks the leader robot's motion under stochastic communication delays.

The interaction between agent and environment is decomposed into the following components:

- **State** $\mathbf{s}_k \in \mathcal{S}$: A complete description of the environment at time step k . The state space $\mathcal{S}$ is the set of all possible configurations the environment can occupy.
- **Action** $\mathbf{a}_k \in \mathcal{A}$: A control output selected by the agent at time step k . The action space $\mathcal{A}$ is the set of all admissible control outputs available to the agent.
- **Policy** π : The agent's decision-making strategy. A policy can be deterministic, $\mathbf{a}_k = \pi(\mathbf{s}_k)$, or stochastic, $\mathbf{a}_k \sim \pi(\cdot | \mathbf{s}_k)$. The policy is the function that the agent optimizes during training.
- **Transition Probability** $\mathcal{P}(\mathbf{s}_{k+1} | \mathbf{s}_k, \mathbf{a}_k)$: The probability of transitioning to state $\mathbf{s}_{k+1}$ given current state $\mathbf{s}_k$ and action $\mathbf{a}_k$. This function characterizes the environment dynamics (i.e., the robot physics and delay patterns).
- **Reward** $r_k = \mathcal{R}(\mathbf{s}_k, \mathbf{a}_k)$: A scalar signal evaluating the immediate quality of action $\mathbf{a}_k$ in state $\mathbf{s}_k$. This signal guides the agent toward desirable tracking behavior.
- **Discount Factor** $\gamma \in [0, 1)$: A parameter determining the importance of future rewards. A value close to 1 emphasizes long-term cumulative return, which is essential for stable trajectory tracking.

Markov Decision Process

The components introduced above are formally unified as a Markov Decision Process (MDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$. The defining characteristic of an MDP is the *Markov property*: the next state depends only on the current state and action, not on the history of prior states. Formally:

$$\mathcal{P}(s_{k+1}|s_k, a_k, s_{k-1}, a_{k-1}, \dots) = \mathcal{P}(s_{k+1}|s_k, a_k). \quad (2.24)$$

This property implies full state observability—the agent must have access to the complete sufficient statistic s_k at each time step.

A standard MDP proceeds as follows: at each discrete time step k , the agent observes the current state s_k , selects an action a_k according to its policy $\pi(a_k|s_k)$, receives a reward $r_k = \mathcal{R}(s_k, a_k)$, and transitions to the next state s_{k+1} according to $\mathcal{P}(s_{k+1}|s_k, a_k)$. The agent’s objective is to find a policy that maximizes the expected cumulative discounted reward:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_k \right]. \quad (2.25)$$

Value Functions

Value functions provide a mechanism to evaluate the quality of a policy. Rather than executing a policy indefinitely to measure its cumulative reward, value functions estimate the expected long-term return, enabling the agent to compare policies and identify improvements without full rollouts.

The *state-value function* $V^\pi(s)$ evaluates the quality of policy π by quantifying the expected cumulative discounted reward when starting from state s and following π thereafter:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{i=0}^{\infty} \gamma^i r_{k+i} \mid s_k = s \right]. \quad (2.26)$$

The *action-value function* (Q-function) $Q^\pi(s, a)$ evaluates policy π by quantifying the expected cumulative discounted reward when taking action a in state s , then following π thereafter:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{i=0}^{\infty} \gamma^i r_{k+i} \mid s_k = s, a_k = a \right]. \quad (2.27)$$

The Q-function plays a central role in policy improvement: given a state s , the agent can compare $Q^\pi(s, a)$ across different actions to identify which yields the highest expected return. The state-value function, in contrast, is primarily used for policy evaluation—assessing the overall quality of the current policy from a given state.

These two functions are related through the expectation over the policy’s action distribution:

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot|s)} [Q^\pi(s, a)]. \quad (2.28)$$

This relationship illustrates that V^π summarizes the quality of policy π at state s by averaging over possible actions, whereas Q^π provides specific utility estimates for individual actions.

Policy Learning

In robot control tasks, the action space is often continuous (e.g., joint torques), preventing the enumeration of all possible actions. Policy-based methods address this by directly parameterizing the policy π with a function approximator, typically a neural network with parameters θ .

For continuous action spaces, the network outputs the parameters of a probability distribution—commonly a Gaussian—from which actions are sampled:

$$a \sim \pi_\theta(\cdot|s) = \mathcal{N}(\mu_\theta(s), \Sigma_\theta(s)). \quad (2.29)$$

The learning objective is to find the optimal parameters θ^* that maximize the expected cumulative discounted reward:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{k=0}^{\infty} \gamma^k r_k \right], \quad (2.30)$$

where $\tau = (s_0, \mathbf{a}_0, s_1, \mathbf{a}_1, \dots)$ denotes a trajectory sampled under policy π_θ .

Policy Gradient

To optimize $J(\theta)$, we employ gradient ascent. The *Policy Gradient Theorem* provides a tractable expression for the gradient of the objective with respect to the policy parameters:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{k=0}^{\infty} \nabla_\theta \log \pi_\theta(\mathbf{a}_k | s_k) Q^{\pi_\theta}(s_k, \mathbf{a}_k) \right]. \quad (2.31)$$

Intuitively, the term $\nabla_\theta \log \pi_\theta(\mathbf{a}_k | s_k)$ represents the direction in parameter space that increases the probability of action $\mathbf{a}_k$, while the scalar $Q^{\pi_\theta}(s_k, \mathbf{a}_k)$ acts as a weight: actions yielding higher expected returns are reinforced more strongly.

Surrogate Loss Function

In practice, modern automatic differentiation libraries minimize a loss function rather than maximizing an objective directly. The surrogate policy loss is defined as:

$$\mathcal{L}_\pi(\theta) = -\mathbb{E}_{s_k, \mathbf{a}_k \sim \mathcal{D}} [\log \pi_\theta(\mathbf{a}_k | s_k) \hat{Q}(s_k, \mathbf{a}_k)], \quad (2.32)$$

where $\hat{Q}$ is an estimate of the Q-function. Minimizing this negative log-likelihood weighted by the Q-value effectively performs the policy gradient update.

Value Function Approximation

As seen in Equation (2.31), policy optimization requires an estimate of the Q-function. Computing the true expected cumulative reward via Monte Carlo rollouts is data-inefficient and has high variance. To address this, we employ *Value Function Approximation*, where the Q-function is parameterized by a neural network (the Critic) with parameters ϕ , denoted as $Q_\phi(s, \mathbf{a})$.

The critic is trained using *Temporal Difference (TD) learning*, which relies on the recursive Bellman expectation equation:

$$Q^\pi(s_k, \mathbf{a}_k) = \mathbb{E}_{s_{k+1} \sim \mathcal{P}} [r_k + \gamma \mathbb{E}_{\mathbf{a}_{k+1} \sim \pi} [Q^\pi(s_{k+1}, \mathbf{a}_{k+1})]]. \quad (2.33)$$

The objective is to minimize the Bellman residual, typically formalized as the Mean Squared Bellman Error (MSBE):

$$\mathcal{L}_Q(\phi) = \mathbb{E}_{\mathcal{D}} \left[(Q_\phi(s_k, \mathbf{a}_k) - y_k)^2 \right], \quad (2.34)$$

where $y_k = r_k + \gamma Q_\phi(s_{k+1}, \mathbf{a}_{k+1})$ serves as the bootstrapping target. By minimizing this loss, the network Q_ϕ converges toward the true action-value function of the current policy.

Partially Observable Markov Decision Process

The MDP framework described above relies on the critical assumption of *full observability*, where the agent has access to the exact system state s_k at every time step. In teleoperation tasks with communication delays, this assumption is violated: the agent receives observations that are delayed, noisy, or incomplete.

Such scenarios are formalized as a Partially Observable Markov Decision Process (POMDP). A POMDP extends the standard MDP tuple to include:

- **Observation Space $\mathcal{O}$:** The set of possible observations the agent can receive.
- **Observation Function $\mathcal{Z}(\mathbf{o}_k | \mathbf{s}_k, \mathbf{a}_{k-1})$:** The probability of receiving observation $\mathbf{o}_k$ given the underlying state $\mathbf{s}_k$.

In a POMDP, the optimal action cannot be determined solely from the current observation $\mathbf{o}_k$ because it does not uniquely identify the state (state aliasing). Instead, the decision must be based on the **history** of past interactions:

$$H_k = (\mathbf{o}_0, \mathbf{a}_0, \mathbf{o}_1, \mathbf{a}_1, \dots, \mathbf{a}_{k-1}, \mathbf{o}_k). \quad (2.35)$$

2.7 Soft Actor-Critic Algorithm

Soft Actor-Critic (SAC) [9] is a state-of-the-art deep reinforcement learning algorithm that instantiates the maximum entropy framework within an off-policy actor-critic architecture. By combining sample-efficient off-policy updates with stable stochastic policy optimization, it addresses the brittleness often observed in deterministic algorithms like DDPG.

Actor-Critic Architecture

SAC relies on the Actor-Critic framework [10], which decomposes the learning process into two distinct components:

- **Actor $\pi_\theta(\mathbf{a} | \mathbf{s})$:** The policy network with parameters θ . Its role is to select actions that maximize the expected return. In SAC, this policy is stochastic and outputs the parameters of a distribution (e.g., Gaussian mean and variance).
- **Critic $Q_\phi(\mathbf{s}, \mathbf{a})$:** The value network with parameters ϕ . Its role is to evaluate the quality of the action selected by the actor. It provides a low-variance learning signal (the gradient of the Q-value) to update the policy parameters.

Maximum Entropy Reinforcement Learning

Standard RL objectives maximize expected cumulative reward, often leading to deterministic policies that exploit known high-reward actions but fail to explore efficiently. To address this, Maximum Entropy RL augments the reward with an entropy bonus. The objective is to maximize:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{k=0}^{\infty} \gamma^k (r_k + \alpha \mathcal{H}(\pi_\theta(\cdot | \mathbf{s}_k))) \right], \quad (2.36)$$

where $\mathcal{H}(\pi_\theta(\cdot | \mathbf{s}_k)) = -\mathbb{E}_{\mathbf{a} \sim \pi_\theta} [\log \pi_\theta(\mathbf{a} | \mathbf{s}_k)]$ is the entropy of the policy at state $\mathbf{s}_k$. The temperature parameter $\alpha > 0$ determines the relative importance of the entropy term, effectively controlling the stochasticity of the optimal policy.

Soft Actor-Critic

Soft Actor-Critic (SAC) introduces three key modifications to improve stability and sample efficiency:

Twin Q-Networks.

To mitigate the overestimation bias common in Q-learning, SAC maintains two independent critic networks, Q_{ϕ_1} and Q_{ϕ_2} . The minimum of the two estimates is used for both policy optimization and target computation:

$$Q_{\min}(\mathbf{s}, \mathbf{a}) = \min(Q_{\phi_1}(\mathbf{s}, \mathbf{a}), Q_{\phi_2}(\mathbf{s}, \mathbf{a})). \quad (2.37)$$

Soft Bellman Equation.

The target value calculation incorporates the entropy of the next action to remain consistent with the maximum entropy objective:

$$y_k = r_k + \gamma (Q_{\min}(s_{k+1}, \mathbf{a}_{k+1}) - \alpha \log \pi_{\theta}(\mathbf{a}_{k+1}|s_{k+1})), \quad (2.38)$$

where $\mathbf{a}_{k+1}$ is sampled from the current policy $\pi_{\theta}(\cdot|s_{k+1})$.

Actor Objective.

The actor is updated to maximize the expected soft Q-value:

$$J_{\pi}(\theta) = \mathbb{E}_{s_k \sim \mathcal{D}, \mathbf{a}_k \sim \pi_{\theta}} [Q_{\min}(s_k, \mathbf{a}_k) - \alpha \log \pi_{\theta}(\mathbf{a}_k|s_k)]. \quad (2.39)$$

The reparameterization trick is used to backpropagate gradients through the stochastic action sampling.

Automatic Temperature Tuning.

Rather than fixing the temperature α , SAC treats the entropy constraint as a dual optimization problem. The temperature is automatically adjusted to maintain a minimum target entropy $\bar{\mathcal{H}}$ (typically $-\dim(\mathcal{A})$) by minimizing the loss:

$$J(\alpha) = \mathbb{E}_{\mathbf{a}_k \sim \pi_{\theta}} [-\alpha (\log \pi_{\theta}(\mathbf{a}_k|s_k) + \bar{\mathcal{H}})]. \quad (2.40)$$

3 Related Work

This chapter reviews delay compensation strategies for teleoperation, tracing the evolution from passivity-based control through model-based prediction to reinforcement learning methods. For each approach, we discuss the key contributions and limitations that motivate the present work.

3.1 Passivity-Based Control

Early approaches to delayed teleoperation drew upon network theory and energy-based stability analysis. Anderson and Spong [11] observed that system instability under time delay arises from the non-passive nature of the communication channel. They introduced the scattering transformation framework, which re-encodes force and velocity signals into wave variables to ensure passive energy flow. Niemeyer and Slotine [12] extended this approach by incorporating adaptive control to handle unknown robot dynamics, providing formal stability guarantees regardless of constant delay magnitude.

However, real-world networks exhibit stochastic, time-varying delays, under which the original scattering transformation can lose passivity. While extensions have been proposed to handle time-varying delays [13], these approaches require additional energy dissipation that further degrades tracking performance. Moreover, the wave transformation inherently trades transparency for stability [14]: the signal encoding attenuates force reflection and reduces tracking fidelity, diminishing the operator’s sense of direct contact with the remote environment.

These limitations motivated researchers to explore predictive approaches that explicitly compensate for delay rather than passively ensuring stability.

3.2 Model-Based Predictive Control

The Smith Predictor [15], originally developed for process control, exemplifies this approach by incorporating a model of the plant and the delay into the feedback loop. The controller simulates the system response forward in time, effectively removing the delay from the closed-loop dynamics. Subsequent work adapted predictive strategies to teleoperation: Sirouspour and Shahdi [16] developed model-based predictors for bilateral teleoperation, and Zheng et al. [17–19] proposed predictor-based delay compensation frameworks for teleoperated ground vehicles, combining model-based and model-free elements to handle varying network delays.

However, these methods share a critical dependency: accurate a priori knowledge of both plant dynamics and delay characteristics. When delays exhibit high stochastic variation or when significant model-plant mismatch exists, predictive performance deteriorates substantially.

These limitations motivated the exploration of learning-based methods that can adapt to unknown dynamics and varying delays without requiring explicit system models.

3.3 Reinforcement Learning Control

Reinforcement learning (RL) offers a promising alternative to classical control by discovering strategies through direct interaction rather than relying on analytical models. However, communication delays transform the standard Markov Decision Process (MDP) into a Partially Observable MDP (POMDP) [20], where the agent must make decisions based on outdated observations. To address this partial observability, two principal paradigms have emerged: *model-free state augmentation* and *model-based prediction*.

3.3.1 Model-Free Approaches (State Augmentation)

Early RL approaches for delayed systems primarily targeted constant, known delays. The foundational technique is *state augmentation*, which concatenates a fixed-length sequence of previous observations and actions to form an expanded input representation. This provides the agent with sufficient historical context to learn implicit patterns correlating past observations with effective control actions. The theoretical basis was established by Altman and Nain [21] in control theory and formalized for RL by Walsh et al. [22], who proved that constant-delay MDPs can be reduced to standard MDPs through state-space augmentation with a buffer of length equal to the delay.

While state augmentation provides an exact solution for constant delays, it introduces fundamental limitations. First, it suffers from the *curse of dimensionality*: the augmented state space grows linearly with delay length, as each additional time step adds a full observation vector to the input. For a system with state dimension n and delay of d steps, the augmented state has dimension $O(nd)$. This drastic increase in input size significantly inflates sample complexity, making learning intractable for long delays.

Second, the fixed history window is ill-suited for the stochastic, time-varying delays encountered in real-world teleoperation. While refinements such as Delay-Resolved DQN [23] and auxiliary short-delay bootstrapping [24] have been proposed, they fundamentally rely on discrete snapshots of history. They lack a mechanism to generate continuous state estimates during intervals where no new packets arrive (jitter), forcing the policy to operate on stale information.

These limitations motivated the exploration of model-based approaches, which explicitly learn to predict the current state from delayed observations.

3.3.2 Model-Based Approaches (Predictive Models)

Model-based methods address partial observability by explicitly forecasting future states. The foundational strategy is *forward dynamics prediction*: given a known system model, the current state can be reconstructed by iteratively simulating the sequence of pending actions starting from the last received observation. Walsh et al. [22] formalized this as Model-Based Simulation (MBS), and it was subsequently integrated with RL to bridge the temporal gap between observation and action [25].

In complex environments where analytical models are unavailable, recent approaches employ learned neural network predictors. The direct adaptation of MBS to learned models is Action-Buffer based State Prediction (ABSP) [26], where a neural model re-simulates the entire queued action sequence at every time step. While conceptually sound, this naive rollout strategy is computationally prohibitive for real-time control.

To address this inefficiency, Chen et al. [27] integrated multi-step delays directly into the trajectory sampling process. A more recent advancement is the Predictive Model Delay Correction (PMDC) framework [7]. To overcome the cost of ABSP, McCutcheon and Fallah introduced State-Buffer based State Prediction (SBSP), which maintains a sliding buffer of predicted future states and updates it incrementally. This $O(1)$ update complexity enables real-time operation without sacrificing multi-step prediction accuracy. PMDC combines this neural predictor with RL to adaptively tune PD controller gains, representing the current state-of-the-art for delay-compensated teleoperation.

Limitations of Existing Methods

Despite these advances, state-of-the-art methods like PMDC exhibit critical limitations when applied to the stochastic, heavy-tailed delay distributions targeted in this work:

- **Delay-Invariant Policy:** Methods relying on implicit history representations without explicit delay information converge to policies optimized for the *average* delay, failing to adapt to instantaneous latency variations. During high-delay spikes, this leads to overshoot and potential safety violations.
- **Discontinuous State Estimation:** One-step predictors update estimates only upon packet arrival, lacking autoregressive mechanisms for intermediate state inference. During extended delays, estimates

degenerate to zero-order holds, causing discontinuous target jumps and control transients when new observations arrive.

- **Cascaded Error Propagation:** Two-stage approaches that separate state prediction from control create a performance bottleneck: prediction errors propagate directly to the controller, which then precisely tracks erroneous targets. Additionally, linear PD structures cannot execute the nonlinear compensation required during sudden delay variations.

These limitations motivate the end-to-end architecture presented in Chapter 4.

4 Methodology

This chapter details the proposed end-to-end reinforcement learning framework designed to address the limitations identified in Chapter 3.

4.1 POMDP Formulation

We formulate the teleoperation control task as a POMDP, instantiating the general framework introduced in Section 2.6. In our setting, the POMDP tuple $(\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, \mathcal{R}, \Omega, \gamma)$ is defined as follows:

- **State space $\mathcal{S}$:** The joint positions and velocities of both leader and follower robots.
- **Action space $\mathcal{A}$:** The joint torques applied to the follower robot.
- **Observation space $\mathcal{O}$:** The follower state, a history of delayed leader packets with explicit delay magnitudes, and past actions.
- **Transition function $\mathcal{T}$:** The robot dynamics governing state evolution.
- **Reward function $\mathcal{R}$:** A combination of tracking accuracy, control effort, and smoothness objectives.
- **Observation function Ω :** The stochastic delay $d_s^k \sim p(d_s)$ that determines which past leader state is observed.
- **Discount factor γ :** The temporal discounting for future rewards.

The following subsections define each component in detail.

4.1.1 State Space

In an ideal teleoperation system without communication delays, the control problem can be formulated as a standard Markov Decision Process (MDP). The true system state at step k comprises the physical configuration of both robots:

$$s_k = [\mathbf{q}_l^{k\top}, \dot{\mathbf{q}}_l^{k\top}, \mathbf{q}_f^{k\top}, \dot{\mathbf{q}}_f^{k\top}]^\top \in \mathbb{R}^{4n}, \quad (4.1)$$

where n is the number of joints, $\mathbf{q}_l^k, \dot{\mathbf{q}}_l^k \in \mathbb{R}^n$ are the leader joint positions and velocities, and $\mathbf{q}_f^k, \dot{\mathbf{q}}_f^k \in \mathbb{R}^n$ are the follower joint positions and velocities. Under this ideal assumption, the state s_k is fully observable and satisfies the Markov property: the optimal action depends only on the current state, not on the history of past observations.

However, in practice, communication delays break this assumption. The follower state $s_f^k = [\mathbf{q}_f^{k\top}, \dot{\mathbf{q}}_f^{k\top}]^\top$ is subject to a constant observation delay d_o , but since d_o is deterministic and known, it can be compensated through local state estimation. In contrast, the leader state $s_l^k = [\mathbf{q}_l^{k\top}, \dot{\mathbf{q}}_l^{k\top}]^\top$ is subject to stochastic network delay d_s^k and cannot be directly observed.

4.1.2 Observation Space

To address this partial observability, we construct an augmented observation vector $\mathbf{o}_k$ that captures temporal context. At each time step k , the agent receives:

$$\mathbf{o}_k = \left[\bar{\mathbf{s}}_f^{k\top}, \mathcal{Z}_k^\top, \mathcal{H}_\tau^{k\top}, \bar{\boldsymbol{\tau}}_{k-1}^\top \right]^\top \in \mathbb{R}^{n_o}, \quad (4.2)$$

where the components are defined as follows:

- $\bar{\mathbf{s}}_f^k \in \mathbb{R}^{14}$ is the current follower state, normalized using pre-computed mean and standard deviation statistics $(\boldsymbol{\mu}, \boldsymbol{\sigma})$ derived from the expert dataset
- $\mathcal{Z}_k = \{\mathbf{z}^{(i)}\}_{i=1}^T$ is a history sequence of length $T = 80$, containing the most recent packets received from the leader
- $\mathcal{H}_\tau^k = \{\bar{\boldsymbol{\tau}}_{k-j}\}_{j=1}^M$ is the history of the past $M = 50$ actions, ensuring the policy is conditioned on its own previous decisions.
- $\bar{\boldsymbol{\tau}}_{k-1} \in \mathbb{R}^n$ is the action taken at the previous step, explicitly provided to encourage control smoothness.

Crucially, each frame $\mathbf{z}^{(i)}$ in the history sequence $\mathcal{Z}_k$ is a composite vector containing the delayed leader state, the follower state *at that historical moment*, and the explicit delay magnitude:

$$\mathbf{z}^{(i)} = \left[\bar{\mathbf{q}}_l^{(i)}, \bar{\mathbf{q}}_f^{(i)}, \bar{d}_s^{(i)}, \bar{\mathbf{q}}_f^{(i)}, \bar{\mathbf{q}}_f^{(i)}, \bar{\boldsymbol{\tau}}^{(i)} \right]^\top \in \mathbb{R}^{36}. \quad (4.3)$$

where the first 15 dimensions (leader state + delay) are processed by the LSTM encoder, and all 36 dimensions provide context for the observation sequence. The normalized delay $\bar{d}_s^{(i)} = d_s^{(i)} / d_{norm}$ indicates the temporal offset of each observation, enabling the predictor to adapt its prediction horizon to instantaneous delay conditions. Constructing a belief state from such interaction history is theoretically sufficient to recover optimal policies in delayed environments.

4.1.3 Action Space

The policy outputs joint torques applied directly to the follower robot:

$$\boldsymbol{\tau}_k \in \mathcal{A} = \left\{ \boldsymbol{\tau} \in \mathbb{R}^n : |\tau^j| \leq \tau_{\max}^j, j = 1, \dots, n \right\}. \quad (4.4)$$

The network produces normalized actions $\tilde{\boldsymbol{\tau}}_k \in [-1, 1]^n$ via tanh activation, which are scaled to physical torques:

$$\boldsymbol{\tau}_k = \boldsymbol{\tau}_{\max} \odot \tilde{\boldsymbol{\tau}}_k, \quad (4.5)$$

where $\boldsymbol{\tau}_{\max} \in \mathbb{R}^n$ contains the torque limits for each joint and $\odot$ denotes element-wise multiplication.

This direct torque formulation bypasses traditional PD control structures, allowing the policy to learn nonlinear control strategies that implicitly compensate for both tracking errors and gravitational effects.

4.2 Reward Function

The reward function encodes the control objective: accurate trajectory tracking with minimal control effort and smooth motion. At each step k :

$$r_k = w_p \cdot r_k^{pos} + w_v \cdot r_k^{vel} - w_u \cdot r_k^{effort} - w_s \cdot r_k^{smooth}, \quad (4.6)$$

where each component is defined as follows.

Position Tracking. The primary objective is minimizing position error between leader and follower:

$$r_k^{pos} = \exp\left(-\alpha_p \|\mathbf{q}_l^k - \mathbf{q}_f^k\|_2\right). \quad (4.7)$$

The exponential formulation bounds values in $[0, 1]$, preventing reward explosion during early training when errors are large. A linear or quadratic penalty would produce unbounded negative rewards for large errors, destabilizing the learning process.

Velocity Tracking. Velocity matching ensures the follower not only reaches the correct position but also moves with the correct speed:

$$r_k^{vel} = \exp\left(-\alpha_v \|\dot{\mathbf{q}}_l^k - \dot{\mathbf{q}}_f^k\|_2\right). \quad (4.8)$$

Without this term, the policy may achieve low position error through aggressive corrections that overshoot and oscillate around the target, rather than smooth trajectory following.

Control Effort. A quadratic penalty discourages excessive torque:

$$r_k^{effort} = \frac{1}{n} \sum_{j=1}^n (\tau_k^j)^2. \quad (4.9)$$

Without effort regularization, the policy may learn to apply maximum torque continuously, causing actuator saturation, energy waste, and potential hardware damage in real-world deployment.

Smoothness. A penalty on action changes promotes smooth control signals:

$$r_k^{smooth} = \|\boldsymbol{\tau}_k - \boldsymbol{\tau}_{k-1}\|_2. \quad (4.10)$$

Without this term, the policy may exhibit high-frequency torque oscillations that are undesirable for physical systems due to mechanical stress and vibration. This is particularly important under stochastic delays, where discontinuous observations can induce abrupt control changes.

4.3 Termination Criteria

To ensure training stability and safety, episodes terminate early under the following conditions:

Divergence. If the position error exceeds a safety threshold:

$$\|\mathbf{q}_l^k - \mathbf{q}_f^k\|_2 > \epsilon_{div}, \quad (4.11)$$

the episode terminates with penalty r_{div} added to the final reward. This criterion serves two purposes: (1) it prevents the simulator from entering physically unrealistic states where the robot configuration diverges significantly from the feasible workspace, and (2) it provides a strong negative signal that discourages the policy from entering unrecoverable error states. Without this termination, the policy may continue accumulating experience from diverged states that are irrelevant to the actual control task.

Numerical Instability. If the simulation detects non-finite values (NaN or Inf) in joint states, the episode terminates immediately with penalty r_{crash} . Such values typically arise from numerical overflow in the physics engine when joint velocities or accelerations become excessively large. Without this safeguard, corrupted states would propagate through the replay buffer, contaminating the training data and causing the neural network to produce unstable outputs.

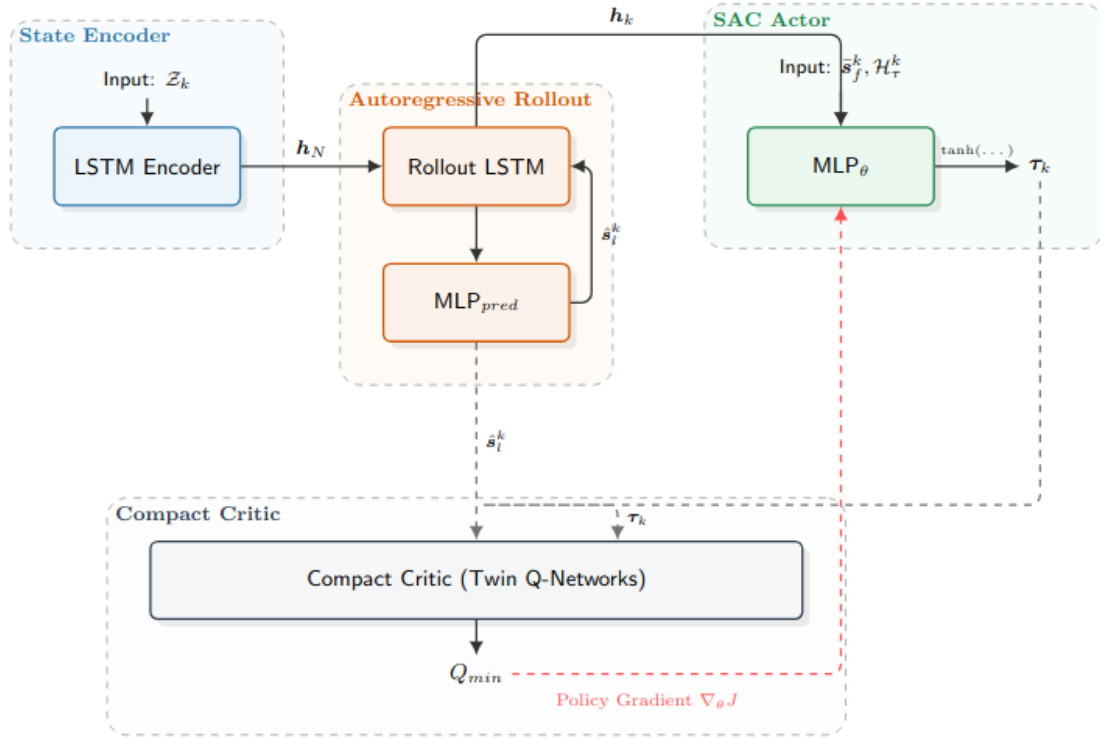


Figure 4.1 End-to-end network architecture. The LSTM encoder processes the delayed observation sequence $\mathcal{Z}_k$, where each input includes the normalized delay magnitude $\bar{d}_s^{(i)}$. The autoregressive rollout uses the encoder’s hidden state to predict intermediate states for H_{ar} steps, producing the estimated current leader state $\hat{s}_l^k$ and latent representation h_k . The SAC policy combines the follower state s_f^k , latent embedding h_k , and action history $\mathcal{H}_\tau^k$ to output joint torques τ_k .

Time Limit. Episodes that do not trigger early termination truncate after T_{max} steps. This fixed horizon ensures consistent episode lengths for batch training and prevents indefinitely long episodes that could bias the learned policy toward short-term behaviors. The truncation is handled as a timeout rather than a terminal state, meaning the bootstrap value is computed from the final state rather than set to zero.

Remark. The reward computation uses the true leader state s_l^k , which is available in simulation during training but not at deployment. This is standard practice in sim-to-real reinforcement learning: the simulator provides privileged information for reward computation, while the policy learns to achieve the objective using only the delayed observation $\mathbf{o}_k$.

4.4 Network Architecture

The proposed architecture integrates a recurrent state estimator with a policy network, designed to handle variable communication delays and high-dimensional history inputs. Figure 4.1 illustrates the complete architecture, which consists of two main components: an LSTM-based state predictor that estimates the current leader state from delayed observations, and a SAC policy network that maps the estimated state to control torques.

4.4.1 LSTM-Based State Predictor

The state predictor recovers the current leader state from a sequence of delayed observations using an LSTMCell with autoregressive rollout.

Encoding Phase

The encoder receives the leader observation sequence $\mathcal{Z}_k^{leader} = \{\mathbf{z}_{leader}^{(i)}\}_{i=1}^N$ where:

$$\mathbf{z}_{leader}^{(i)} = [\bar{\mathbf{q}}_l^{(i)}, \bar{\mathbf{q}}_l^{(i)}, \bar{d}_s^{(i)}]^\top \in \mathbb{R}^{15}. \quad (4.12)$$

The LSTMCell processes the sequence to produce hidden and cell states:

$$\mathbf{h}_i, \mathbf{c}_i = \text{LSTMCell}(\mathbf{z}_{leader}^{(i)}, (\mathbf{h}_{i-1}, \mathbf{c}_{i-1})), \quad i = 1, \dots, N. \quad (4.13)$$

After encoding, the anchor state and initial delay are extracted from the most recent observation:

$$\hat{\mathbf{q}}_0 = \bar{\mathbf{q}}_l^{(N)}, \quad \hat{\mathbf{v}}_0 = \bar{\mathbf{q}}_l^{(N)}, \quad \delta_0 = \bar{d}_s^{(N)}. \quad (4.14)$$

Autoregressive Rollout Phase

The number of rollout steps is determined by the current delay:

$$H_{ar} = \lceil \delta_0 \cdot \eta_{delay} \rceil, \quad (4.15)$$

where η_{delay} is the delay normalization factor that converts normalized delay to discrete steps.

At each rollout step $j = 0, \dots, H_{ar} - 1$, the network predicts the velocity correction and integrates forward:

$$\mathbf{h}_{j+1}, \mathbf{c}_{j+1} = \text{LSTMCell}([\hat{\mathbf{q}}_j, \hat{\mathbf{v}}_j, \delta_j], (\mathbf{h}_j, \mathbf{c}_j)), \quad (4.16)$$

$$\Delta \mathbf{v}_j = \text{MLP}_{pred}(\mathbf{h}_{j+1}), \quad (4.17)$$

$$\hat{\mathbf{v}}_{j+1} = \hat{\mathbf{v}}_j + \Delta \mathbf{v}_j, \quad (4.18)$$

$$\hat{\mathbf{q}}_{j+1} = \hat{\mathbf{q}}_j + \hat{\mathbf{v}}_j \odot \boldsymbol{\beta}_{dt}, \quad (4.19)$$

$$\delta_{j+1} = \max(\delta_j - \Delta t, 0), \quad (4.20)$$

where MLP_{pred} is a two-layer prediction head, and $\boldsymbol{\beta}_{dt} = (\boldsymbol{\sigma}_{\dot{q}} / \boldsymbol{\sigma}_q) \cdot \Delta t$ is a scaling factor that accounts for the normalization statistics when integrating in normalized space.

A binary mask ensures that rollout only continues while $j < H_{ar}$:

$$m_j = \mathbf{1}[j < H_{ar}], \quad (4.21)$$

and all state updates are gated by this mask to handle variable-length rollouts within a batch.

The final outputs are:

- Predicted leader state: $\hat{\mathbf{s}}_l^k = [\hat{\mathbf{q}}_{H_{ar}}, \hat{\mathbf{v}}_{H_{ar}}]^\top \in \mathbb{R}^{14}$
- Latent representation: $\mathbf{h}_k = \mathbf{h}_{H_{ar}} \in \mathbb{R}^{d_h}$

Similar to latent dynamics models used in model-based reinforcement learning [28], our autoregressive rollout predicts future states in a compact latent space, avoiding the computational overhead of full state-space simulation while maintaining physical consistency through the prediction loss.

4.4.2 SAC Policy Network

Actor Network

The actor π_θ is an MLP that receives a composite input:

$$\mathbf{x}_\pi = [\bar{\mathbf{s}}_f^k, \mathbf{h}_k, \text{flatten}(\mathcal{H}_\tau^k), \bar{\boldsymbol{\tau}}_{k-1}]^\top, \quad (4.22)$$

where $\bar{s}_f^k$ is the normalized follower state, $\mathbf{h}_k$ is the LSTM latent, and the action history provides temporal context for smooth control.

The network outputs parameters of a diagonal Gaussian:

$$\boldsymbol{\mu}_k, \log \boldsymbol{\sigma}_k = \text{MLP}_\theta(\mathbf{x}_\pi), \quad (4.23)$$

where $\log \boldsymbol{\sigma}_k$ is clamped to $[\sigma_{\min}, \sigma_{\max}]$ for numerical stability.

Actions are sampled via the reparameterization trick with tanh squashing:

$$\boldsymbol{\tau}_k = \boldsymbol{\tau}_{\max} \odot \tanh(\boldsymbol{\mu}_k + \boldsymbol{\sigma}_k \odot \boldsymbol{\epsilon}), \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (4.24)$$

The log-probability includes the Jacobian correction for the tanh transform:

$$\log \pi_\theta(\boldsymbol{\tau}_k | \mathbf{o}_k) = \sum_{j=1}^n \left[\log p(\epsilon_j) - \log \left(\tau_{\max}^j (1 - \tanh^2(u_j)) + \epsilon_{\text{num}} \right) \right], \quad (4.25)$$

where $u_j = \mu_j + \sigma_j \epsilon_j$ and ϵ_{num} is a small constant for numerical stability.

Compact Critic Network

To mitigate the curse of dimensionality from high-dimensional history inputs, we employ a compact critic architecture. Instead of processing the full observation (which may exceed 500 dimensions), the critic receives a compressed representation:

$$\mathbf{x}_Q = [\bar{s}_f^k, \mathbf{h}_k, \hat{s}_l^k, \boldsymbol{\tau}_k]^\top \in \mathbb{R}^{d_Q}, \quad (4.26)$$

where $d_Q = 14 + d_h + 14 + n$ is significantly smaller than the full observation dimension. The latent $\mathbf{h}_k$ serves as a sufficient statistic for the observation history.

Twin Q-networks with LayerNorm are used for stability:

$$Q_{\psi_i}(\mathbf{x}_Q) = \text{MLP}_{\psi_i}(\mathbf{x}_Q), \quad i \in \{1, 2\}, \quad (4.27)$$

with the minimum used for policy updates:

$$Q_{\min}(\mathbf{x}_Q) = \min(Q_{\psi_1}(\mathbf{x}_Q), Q_{\psi_2}(\mathbf{x}_Q)). \quad (4.28)$$

4.5 Training Procedure

Training proceeds in two phases: supervised pre-training via behavioral cloning, followed by reinforcement learning fine-tuning. Figure 4.2 illustrates the training pipeline.

4.5.1 Prediction Loss

To enforce physical consistency in the latent space, we apply a supervised prediction loss. The loss compares the final predicted state against ground truth:

$$\mathcal{L}_{\text{pred}} = \|\hat{s}_l^k - s_l^{k,gt}\|^2, \quad (4.29)$$

where $s_l^{k,gt}$ is the true (normalized) leader state at step k , available during training.

4.5.2 Behavioral Cloning

The first phase initializes the encoder and policy using expert demonstrations, a paradigm often termed Demonstration-Augmented Policy Gradient [29], establishing stable behavior before RL exploration.

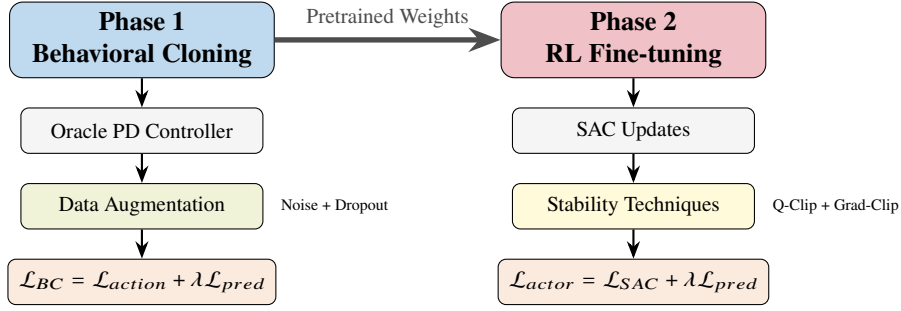


Figure 4.2 Two-phase training pipeline. Phase 1 pretrains the encoder and policy via behavioral cloning using an oracle PD controller with data augmentation (state noise and history dropout). Phase 2 fine-tunes the model using SAC with stability techniques (adaptive Q-clipping, gradient clipping) while maintaining the auxiliary prediction loss $\mathcal{L}_{pred}$.

Expert Data Generation

An oracle dataset is generated using a high-gain PD controller with access to the true (undelayed) leader state:

$$\tau_{expert} = K_p(\mathbf{q}_l^k - \mathbf{q}_f^k) + K_d(\dot{\mathbf{q}}_l^k - \dot{\mathbf{q}}_f^k), \quad (4.30)$$

where K_p and K_d are diagonal gain matrices. Trajectories are collected under stochastic delay conditions matching the target deployment distribution.

Loss Function

The BC loss combines action imitation with state prediction:

$$\mathcal{L}_{BC} = \|\boldsymbol{\mu}_\theta(\mathbf{o}_k) - \bar{\tau}_{expert}\|^2 + \lambda_{aux} \mathcal{L}_{pred}, \quad (4.31)$$

where $\boldsymbol{\mu}_\theta$ is the mean output of the policy (before sampling), $\bar{\tau}_{expert}$ is the normalized expert action, and λ_{aux} weights the auxiliary prediction loss.

Data Augmentation

To mitigate the covariate shift phenomenon inherent in off-policy imitation learning [30], we apply the following augmentations:

- **State Noise:** Gaussian noise $\mathcal{N}(0, \sigma_{aug}^2)$ is added to state inputs during training, forcing the policy to handle minor state estimation errors.
- **History Dropout:** With probability p_{drop} , the action history $\mathcal{H}_\tau^k$ is zeroed out, forcing the policy to rely on the current state and latent embedding rather than memorizing action sequences.

Since standard RL policies often degrade when deployed in environments with non-stationary delays [31], these augmentations prevent overfitting to the expert trajectory distribution and improve generalization to the stochastic dynamics encountered during RL fine-tuning.

4.5.3 RL Fine-tuning

After behavioral cloning, the model is fine-tuned using Soft Actor-Critic.

Combined Loss

To prevent encoder degradation during policy updates:

$$\mathcal{L}_{actor} = \mathcal{L}_{SAC} + \lambda_{aux} \mathcal{L}_{pred}. \quad (4.32)$$

The critic gradient does not flow through the LSTM encoder; the latent $\mathbf{h}_k$ and predicted state $\hat{\mathbf{s}}_t^k$ are detached when computing the policy gradient, ensuring stable representation learning.

Critic Update

Twin Q-networks minimize the soft Bellman residual:

$$\mathcal{L}_Q(\psi_i) = \mathbb{E} \left[\left(Q_{\psi_i}(\mathbf{x}_Q^k) - y_k \right)^2 \right], \quad (4.33)$$

where:

$$y_k = r_k + \gamma \left(Q_{min}(\mathbf{x}_Q^{k+1}) - \alpha \log \pi_\theta(\boldsymbol{\tau}_{k+1} | \mathbf{o}_{k+1}) \right). \quad (4.34)$$

Actor Update

The policy maximizes expected Q-value plus entropy:

$$\mathcal{L}_\pi(\theta) = \mathbb{E} \left[\alpha \log \pi_\theta(\boldsymbol{\tau}_k | \mathbf{o}_k) - Q_{min}(\mathbf{x}_Q^k) \right]. \quad (4.35)$$

Temperature Update

The temperature α is adjusted to maintain target entropy:

$$\mathcal{L}(\alpha) = \mathbb{E} \left[-\alpha \left(\log \pi_\theta(\boldsymbol{\tau}_k | \mathbf{o}_k) + \bar{\mathcal{H}} \right) \right], \quad (4.36)$$

where $\bar{\mathcal{H}} = -n \cdot \delta_e$ with $\delta_e < 1$ to constrain exploration during fine-tuning. The log-temperature is clamped to prevent extreme values.

Training Stability

Adaptive Q-Clipping. Target Q-values are clamped based on running statistics to mitigate value overestimation:

$$y_k \leftarrow \text{clip} \left(y_k, \mu_Q - c\sigma_Q, \mu_Q + c\sigma_Q \right), \quad (4.37)$$

where μ_Q and σ_Q are exponential moving averages of Q-values, and c determines the clipping range.

Gradient Clipping. Gradients are clipped to maximum norm g_{max} to prevent destabilizing updates.

Soft Target Updates. Target networks are updated via Polyak averaging:

$$\bar{\psi} \leftarrow \tau_{soft} \psi + (1 - \tau_{soft}) \bar{\psi}. \quad (4.38)$$

5 Experiments

All experiments employ a seven-degree-of-freedom Franka Panda robot arm integrated with ROS2 Humble middleware.

5.1 Experimental Setup

The teleoperation system architecture deploys the RL agent co-located with the follower robot, resulting in constant action delay $\omega_o^t = \omega_a^t = 50$ ms between the agent’s torque commands and the follower robot’s execution for all t . Stochasticity arises from the network communication between the leader robot and the RL agent, introducing variable observation delay $\omega_s^t \sim \mathcal{U}(\omega_{\min}, \omega_{\max})$. And the sampling period is set as $\Delta t = 4$ ms.

We evaluate three delay configurations shown in Fig. 5.1 as follows:

- **Low delay with low variance:** $d_{\text{obs}} \sim \mathcal{U}(120, 160)$ ms, total: 170–210 ms
- **High delay with low variance:** $d_{\text{obs}} \sim \mathcal{U}(200, 240)$ ms, total: 250–290 ms
- **High delay with high variance:** $d_{\text{obs}} \sim \mathcal{U}(40, 240)$ ms, total: 90–290 ms

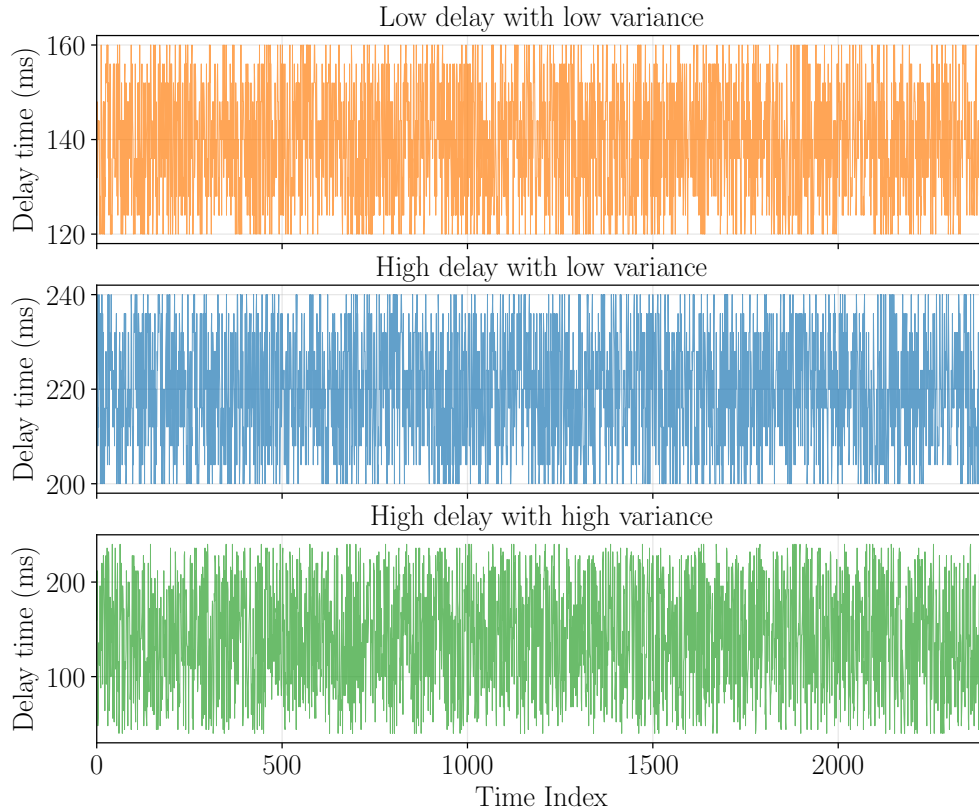


Figure 5.1 State delay patterns in teleoperation tasks. Low delay with low variance (top), high delay with low variance (middle), and high delay with high variance (bottom).

5.2 Evaluation Metrics

To quantify the tracking performance, a desired reference trajectory for the end-effector is commanded to the leader robot, which the follower must replicate under delay. The reference trajectory in the task space is defined as:

$$\mathbf{x}_l^{\text{ref}}(k) = \begin{bmatrix} 0.2 \sin(3k) \\ 0.2 \sin(4k + \frac{\pi}{2}) \\ 0.02 \sin(k) \end{bmatrix}. \quad (5.1)$$

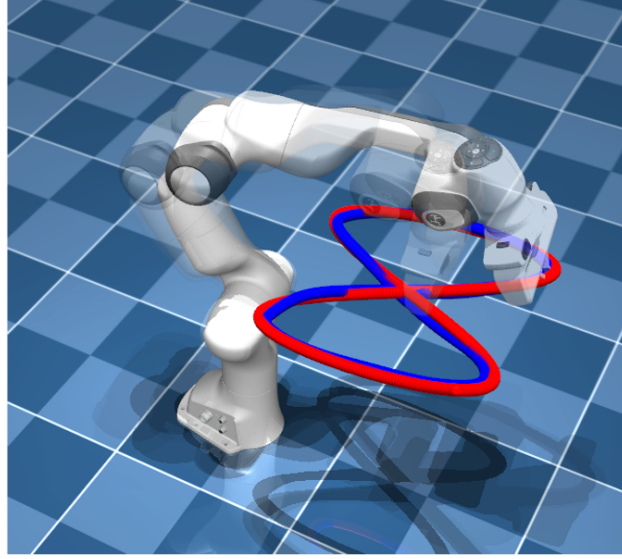


Figure 5.2 Spatial trajectories under high-variance delay. Red: leader trajectory. Blue: follower trajectory.

The primary control objective is to achieve high-fidelity end-effector tracking between the leader and follower robots. Let $\mathbf{x}_l^k = \mathbf{f}(\mathbf{q}_l^k) \in \mathbb{R}^m$ and $\mathbf{x}_f^k = \mathbf{f}_{kin}(\mathbf{q}_f^k) \in \mathbb{R}^m$ denote the end-effector positions of the leader and follower at step k , where $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is the forward kinematics mapping.

Since the human operator commands end-effector motion, the task-space tracking error is the fundamental performance metric:

$$\mathbf{e}_x^k = \mathbf{x}_l^k - \mathbf{x}_f^k. \quad (5.2)$$

However, because the leader's joint positions $\mathbf{q}_l^k$ are transmitted directly over the network, the control policy operates in joint space. For kinematically identical leader and follower robots, minimizing the joint-space error

$$\mathbf{e}_q^k = \mathbf{q}_l^k - \mathbf{q}_f^k \quad (5.3)$$

implicitly minimizes the task-space error $\mathbf{e}_x^k$, since $\mathbf{e}_x^k = \mathbf{0}$ when $\mathbf{e}_q^k = \mathbf{0}$.

The goal is to design a control policy that minimizes $\|\mathbf{e}_q^k\|$ under stochastic communication delays, thereby achieving accurate end-effector tracking.

5.3 Result Analysis

5.3.1 Experimental Protocol and Baseline Definitions

To rigorously evaluate the efficacy of the proposed delay compensation mechanisms, we conduct a comparative analysis of four distinct control strategies. These models were selected to represent the full spectrum of current approaches, ranging from classical control to state-of-the-art (SOTA) learning-based methods:

1. **Vanilla PD (Lower Bound):** A classical controller operating on delayed observations, serving as the baseline for uncompensated performance.
2. **SBSP (Model-Based SOTA):** The Predictive Model Delay Compensation framework [7], representing the current standard for model-based state prediction.
3. **ASAC (Model-Free SOTA):** Augmented Soft Actor-Critic [23], representing standard history-stacking approaches without explicit delay adaptation.
4. **Ours (Proposed Method):** The complete end-to-end framework integrating the LSTM encoder with reinforcement learning.

5.3.2 Performance on Fixed Trajectory

In this initial evaluation phase, all policies are trained and tested on the fixed reference trajectory defined in Eq. (5.1). By constraining the spatial task to a known, repetitive trajectory, we isolate network latency as the primary independent variable. This controlled setting eliminates spatial generalization as a confounding factor, allowing us to strictly assess how each architecture handles temporal delays and jitter.

Figure 5.3 illustrates the temporal evolution of the task-space tracking error $\|e_x\|_2$ (as defined in Eq. 5.2) for the fixed reference trajectory. The results are stratified across the three delay configurations detailed in Section 5.1.

In the low delay with low variance regime (Fig. 5.3a), the proposed E2E-RL method demonstrates superior tracking fidelity, achieving the lowest mean tracking error of $\mu = 0.022$ m. This outperforms the model-based SBSP baseline ($\mu = 0.033$ m), suggesting that our end-to-end policy can learn a more effective compensation strategy than explicit state prediction alone. In contrast, the Vanilla PD controller fails to stabilize ($\mu = 0.140$ m), and the model-free ASAC baseline exhibits significant oscillation ($\mu = 0.061$ m), confirming that standard history stacking is insufficient for precise control even under mild delay conditions.

Performance under High-Delay Conditions

As latency increases to the high-delay setting (Fig. 5.3b), baseline methods show significant degradation—SBSP error increases to $\mu = 0.040$ m and ASAC to $\mu = 0.076$ m—while our method maintains near-constant performance ($\mu = 0.022$ m).

The baseline failures reveal fundamental architectural limitations. ASAC relies on a fixed-length history stack that becomes insufficient as delay increases, requiring window sizes that scale as $O(d_s^k)$ and leading to prohibitive input dimensionality. SBSP employs one-shot prediction that estimates a single state correction per packet, leaving the controller without state updates between arrivals.

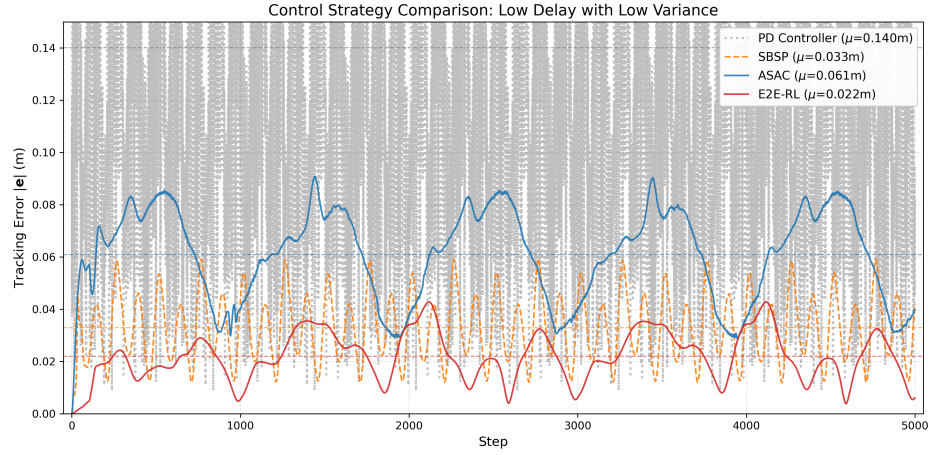
Our architecture overcomes both limitations through continuous autoregressive rollout. The LSTM encoder propagates predicted states forward at every timestep, independent of packet arrival timing, providing uninterrupted state estimates regardless of delay magnitude.

Robustness to High-Variance Jitter

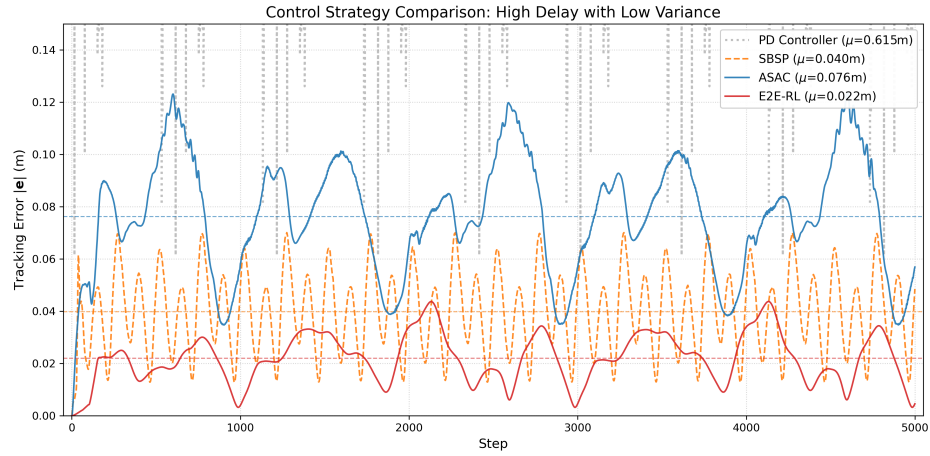
The critical advantage of the proposed architecture is demonstrated in the high-variance delay scenario (Fig. 5.3c). This configuration simulates extreme network jitter where the observation delay d_s^k fluctuates uniformly between 90 ms and 290 ms.

Under these stochastic conditions, the baseline methods exhibit distinct failure patterns. SBSP shows pronounced oscillatory behavior—its tracking error rapidly alternates between low and high values throughout the trajectory. This jittering reflects the limitation of one-shot prediction: each newly arrived packet with a different delay magnitude produces a discontinuous state estimate, causing abrupt corrections in the control signal. ASAC exhibits large, slow-varying error fluctuations, indicating failure to maintain consistent tracking without explicit delay modeling.

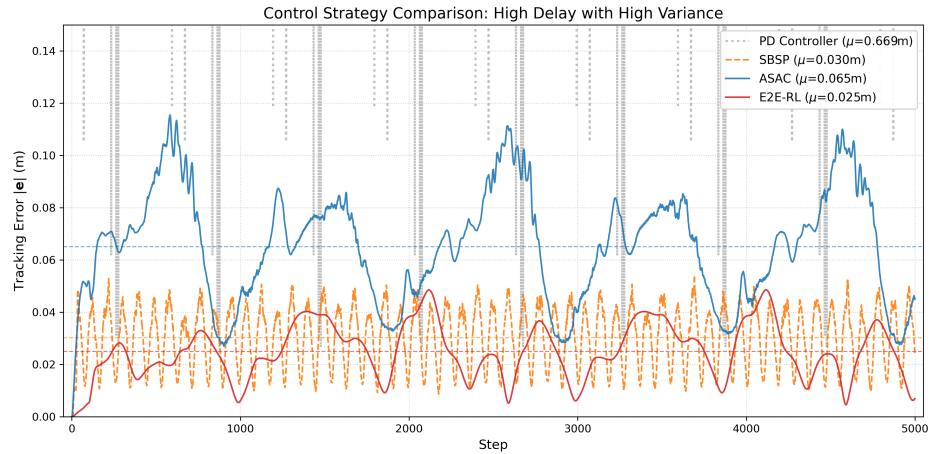
In contrast, our method maintains smooth, low-amplitude tracking error ($\mu = 0.022$ m) comparable to stable delay cases. This robustness stems from two architectural features:



(a) Low Delay with Low Variance (170–210 ms)



(b) High Delay with Low Variance (250–290 ms)



(c) High Delay with High Variance (90–290 ms)

Figure 5.3 Comparative analysis of end-effector tracking error $\|e_x\|_2$ (m) across three delay profiles on the fixed reference trajectory. From top to bottom: (a) low delay, low variance (b) high delay, low variance and (c) high-variance stochastic delay. The proposed method (Ours) demonstrates superior consistency compared to baselines, particularly in the high-variance regime.

1. **Explicit Delay Injection:** The instantaneous delay magnitude $\bar{d}_s^{(i)}$ is provided as an input feature, enabling the encoder to dynamically adjust its prediction horizon at every timestep rather than relying on a static mean estimate.
2. **Continuous Autoregressive Rollout:** The LSTM encoder propagates system dynamics forward from the delayed observation to the estimated current state at every control step, providing temporally coherent estimates that absorb delay variability.

This combination enables the policy to generate smooth, delay-compensated control signals even under extreme network jitter.

5.3.3 Generalization to Randomized Trajectories

While the fixed trajectory experiments isolate delay robustness, real-world teleoperation requires the control policy to generalize across the robot’s workspace without overfitting to a specific repetitive motion. To evaluate this spatial generalization capability, we introduce a **Randomized Trajectory Protocol**.

Trajectory Randomization Configuration

To evaluate generalization capability, we generate unique reference trajectories for each evaluation episode by randomizing the workspace center (C), spatial scale (S), and frequency (f). Parameters are sampled uniformly from the ranges defined in Table 5.1.

Table 5.1 Randomized trajectory parameter ranges. Parameters are sampled uniformly $\mathcal{U}(\min, \max)$ for each of $N = 10$ evaluation episodes.

Parameter	Range	Unit
Center X (C_x)	[0.25, 0.35]	m
Center Y (C_y)	[-0.1, 0.1]	m
Scale X/Y ($S_{x,y}$)	[0.15, 0.25]	m
Frequency (f)	[0.05, 0.15]	Hz

All methods are evaluated under the high-variance delay configuration (90–290 ms) across $N = 10$ independent trials, ensuring that reported performance reflects generalization rather than overfitting to a specific path.

Generalization Performance

Figure 5.4 presents the tracking error under randomized spatial conditions.

The results reveal a significant generalization gap in the baseline methods. SBSP suffers notable performance degradation compared to fixed trajectories (mean error increasing from 0.033 m to 0.046 m), and exhibits pronounced jittering behavior as shown in the figure. This oscillatory pattern indicates that its fixed-horizon predictor overfits to predictable trajectory structures and produces discontinuous estimates when the reference geometry changes dynamically. ASAC remains suboptimal ($\mu = 0.059$ m) with large error fluctuations throughout the trajectory.

In contrast, our method maintains smooth, low-amplitude error ($\mu = 0.022$ m), comparable to its performance on fixed trajectories ($\mu = 0.023$ m). This consistency confirms that the delay-adaptive encoder captures the underlying temporal dynamics of the delayed system rather than overfitting to specific spatial patterns.

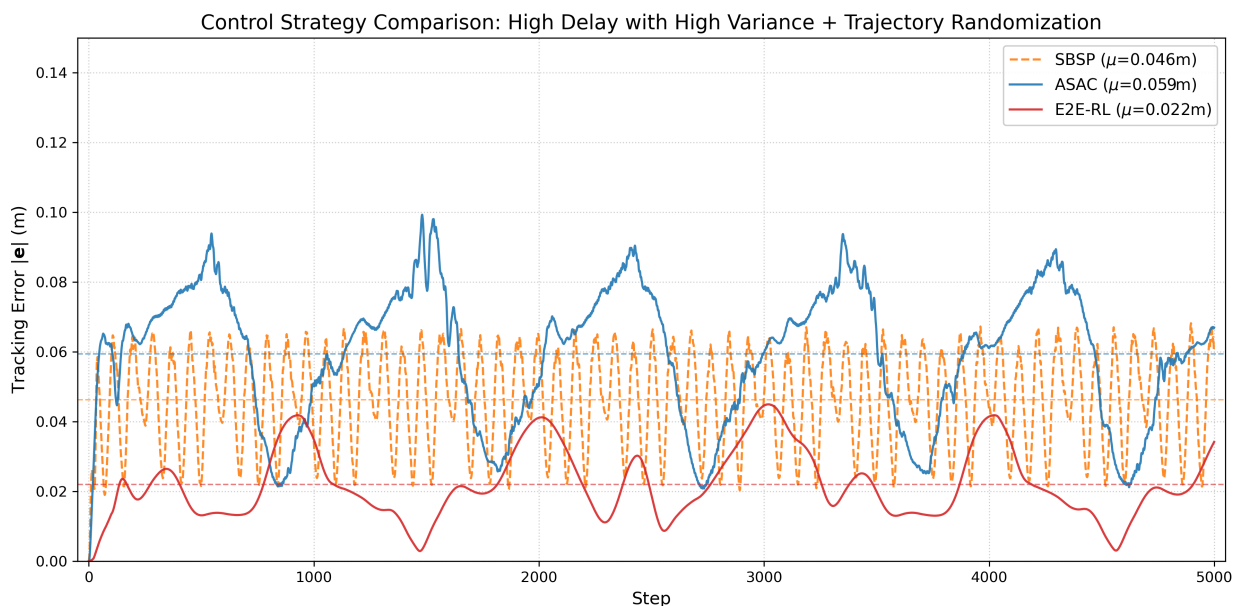


Figure 5.4 Tracking error under randomized trajectories with high-variance delay. E2E-RL (Ours) maintains consistent precision ($\mu = 0.022$ m), while SBSP exhibits oscillatory behavior ($\mu = 0.046$ m) and ASAC shows large error fluctuations ($\mu = 0.059$ m).

5.3.4 Statistical Error Distribution

To complement the time-series analysis, Figure 5.5 presents the statistical distribution of tracking errors across all experimental conditions. The box plots show median (horizontal line), mean (diamond), interquartile range (box), and min/max values (whiskers).

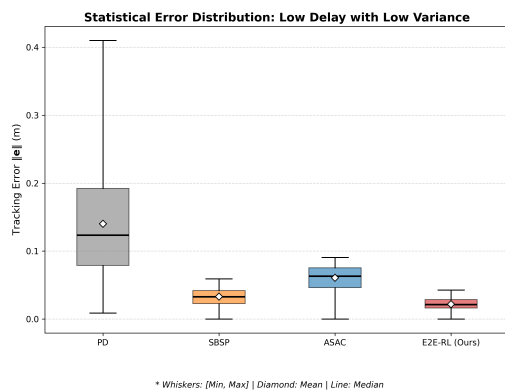
The statistical analysis reveals several key observations:

- **Consistent Low Variance:** E2E-RL maintains the smallest interquartile range across all conditions, indicating stable and predictable tracking behavior regardless of delay configuration.
- **PD Controller Failure:** Under high-delay conditions, the baseline PD exhibits error spread exceeding 1.0 m with extreme outliers, confirming its inability to handle significant latency.
- **SBSP Sensitivity to Randomization:** While SBSP performs competitively on fixed trajectories, its error distribution widens substantially under trajectory randomization, reflecting overfitting to predictable motion patterns.
- **ASAC Instability:** ASAC consistently shows wider error distributions than E2E-RL across all conditions, with higher maximum errors indicating occasional tracking failures.

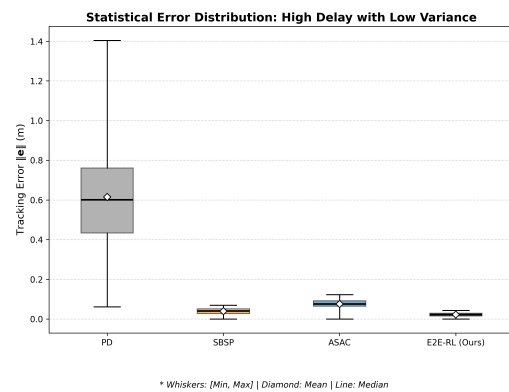
Table 5.2 summarizes the quantitative results across all experimental conditions.

Table 5.2 Mean tracking error (μ) in meters across experimental conditions.

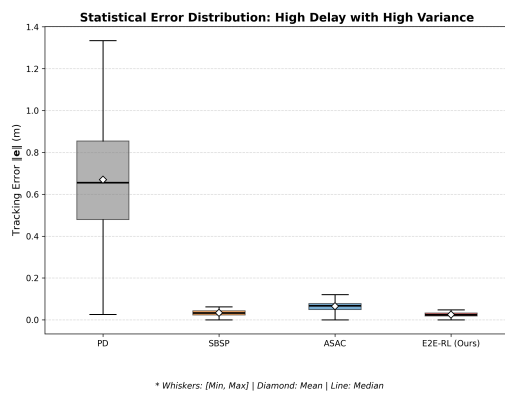
Condition	PD	SBSP	ASAC	E2E-RL (Ours)
Low delay, low var	0.140	0.033	0.061	0.022
High delay, low var	0.610	0.040	0.076	0.022
High delay, high var	0.670	0.033	0.068	0.025
+ Trajectory rand.	—	0.046	0.059	0.022



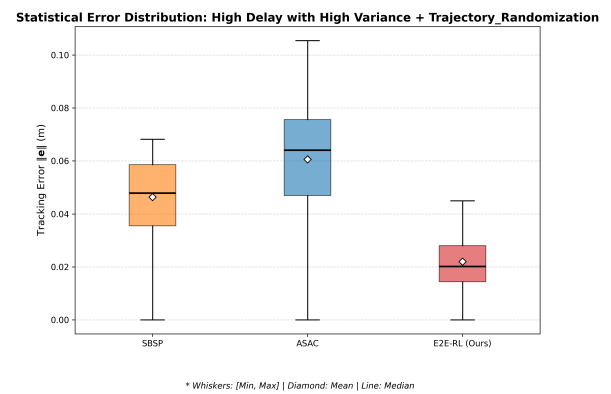
(a) Low delay with low variance



(b) High delay with low variance



(c) High delay with high variance



(d) High variance + trajectory randomization

Figure 5.5 Statistical error distribution across experimental conditions. E2E-RL (Ours) consistently achieves the lowest mean and tightest interquartile range across all scenarios. The PD controller fails under high delay (error exceeding 1.0 m), while SBSP shows increased spread under trajectory randomization.

6 Results and Conclusion

6.1 Discussion

This section examines the architectural design choices, analyzes the limitations of the current framework, and discusses the implications for practical deployment.

6.1.1 Architectural Efficiency and Inductive Bias

While recent advances in sequence modeling have established powerful attention-based architectures, notably the Transformer [32], our framework deliberately employs an LSTM-based [8] state estimator. This design choice is grounded in two specific constraints of high-frequency teleoperation.

Inference Latency Constraint. Transformer architectures scale quadratically $O(N^2)$ with sequence length due to the self-attention mechanism. Although linear-attention variants exist, they often require buffering a history window, introducing computational overhead incompatible with hard real-time requirements of 1 kHz control loops. In contrast, the LSTM processes temporal data recurrently, maintaining a compact hidden state that enables $O(1)$ constant-time inference per step, ensuring strictly bounded latency regardless of history length.

Inductive Bias for Continuous Dynamics. Transformers are designed for discrete sequential data such as language tokens, where long-range dependencies between distant elements are common. In contrast, robot teleoperation involves continuous physical states that evolve smoothly over time—each state is highly correlated with its immediate neighbors. The recurrent structure of LSTM naturally captures this local temporal continuity, making it well-suited for modeling physical dynamics. Transformers, lacking this built-in bias, must learn such continuity from data, requiring more samples to achieve comparable performance on control tasks.

Why Not State Space Models? Recent State Space Models (SSMs) [33] and Mamba [34] offer an attractive middle ground: $O(1)$ inference complexity like LSTMs, combined with improved long-range dependency modeling like Transformers. However, we chose LSTM for the following reasons:

- **Maturity and Stability:** LSTMs are well-understood with decades of research on training dynamics, initialization, and hyperparameter tuning. SSMs and Mamba are relatively new, with less established best practices for control applications.
- **Sufficient Capacity:** Our delay compensation task primarily requires modeling short-term dynamics (tens to hundreds of milliseconds). The limited long-range capacity of LSTMs is not a bottleneck for this application, making the additional complexity of SSMs unnecessary.
- **Implementation Simplicity:** LSTMs are natively supported in all deep learning frameworks with optimized implementations. SSM/Mamba architectures require specialized implementations that may introduce additional deployment complexity.

Future work could explore whether SSM-based encoders provide benefits for delay patterns with longer temporal correlations.

Table 6.1 summarizes the trade-offs between architectural choices.

Table 6.1 Comparison of sequence modeling architectures for real-time control.

Property	LSTM	Transformer	SSM/Mamba
Inference complexity	$O(1)$	$O(N^2)$	$O(1)$
Long-range dependencies	Limited	Strong	Strong
Local temporal bias	Strong	Weak	Moderate
Sample efficiency	High	Low	Moderate
Real-time suitability	High	Low	High
Maturity	High	High	Low

6.1.2 Direct Torque Control versus PD-Based Approaches

A key architectural decision in this work is the use of direct torque output rather than learning PD gain adjustments like previous works. This choice involves a fundamental trade-off between control flexibility and training stability.

Advantages of PD-Based Control. Cascaded architectures that use RL to tune PD gains inherit the stability guarantees of the underlying PD controller. Even when the learned component produces suboptimal outputs, the PD structure ensures that the robot moves toward the desired state with bounded error. This implicit regularization makes training more stable and reduces the risk of catastrophic failures during exploration.

Limitations of PD-Based Control. However, the performance of such approaches is fundamentally bounded by the accuracy of the state predictor. If the estimated target position $\hat{\mathbf{q}}_t^k$ contains errors, the PD controller will faithfully track this erroneous reference, propagating estimation errors into control errors. The cascaded structure prevents the policy from learning compensatory strategies that could mitigate predictor inaccuracies.

Direct Torque Control: Flexibility at a Cost. Direct torque control removes this bottleneck, allowing the policy to learn arbitrary nonlinear control strategies that can implicitly compensate for prediction errors, model mismatches, and unmodeled dynamics. However, this flexibility comes at the cost of training stability due to compounding errors.

Consider a scenario where the policy outputs a torque that deviates by a small amount from the optimal command. This deviation causes the robot to arrive at a state that differs from the intended target. Crucially, this erroneous state may lie outside the distribution of states encountered during training. When the policy must then select the next action from this unfamiliar state, it lacks the experience to make a precise decision, leading to further deviation. This error accumulates over subsequent timesteps: each suboptimal action pushes the robot into increasingly unfamiliar regions of the state space, where the policy becomes progressively less reliable.

This compounding effect—where small torque errors propagate into large state deviations, which in turn induce larger control errors—is the fundamental source of training instability in direct torque control. Unlike PD-based approaches where the controller structure inherently corrects for deviations toward the target, a learned torque policy has no such implicit recovery mechanism. The resulting noisy trajectories generate high-variance gradients that destabilize learning, often causing the policy to diverge entirely.

The Role of Behavioral Cloning. This instability motivates the two-phase training procedure employed in this work. Behavioral cloning initializes the policy in a region of the parameter space that produces stable, smooth control signals. The BC phase effectively transfers the implicit stability of the oracle PD controller into the neural network weights, providing a safe starting point for subsequent RL fine-tuning.

Critically, the data augmentation applied during BC—state noise injection and action history dropout serves a dual purpose. First, it mitigates covariate shift by exposing the policy to perturbed states similar to those encountered during imperfect execution. Second, and more importantly for direct torque control, it prepares the policy for the stochastic perturbations introduced by RL exploration. A policy trained on clean expert data would produce brittle control signals that degrade rapidly when RL adds exploration noise. By injecting noise during BC, the policy learns to produce stable torques even when inputs are slightly corrupted, enabling smoother transition to the RL phase where the policy must tolerate its own exploratory actions.

Without this initialization and augmentation, training a direct torque policy from scratch with RL alone would require extensive reward shaping, conservative exploration strategies, or prohibitively long training times to avoid divergent behaviors.

6.2 Limitations

Despite the demonstrated improvements of our model, several limitations of the proposed framework warrant discussion.

Trajectory Distribution Dependency. The LSTM encoder and policy are trained on a specific distribution of trajectories generated during data collection. If the operator executes motions that deviate significantly from this training distribution, such as sudden direction reversals, high-frequency oscillations, or velocities exceeding the training range—the predictor may fail to accurately estimate the current leader state. Since the RL policy is conditioned on these predictions, degraded estimation accuracy directly translates to degraded control performance. In extreme cases, a sudden, unexpected trajectory change can cause the predictor to output states far from reality, leading the policy to produce inappropriate torques and potentially destabilizing the system.

This limitation is inherent to learning-based approaches: the model can only generalize within the support of its training distribution. Expanding this support requires either collecting more diverse trajectory data or developing online adaptation mechanisms that can adjust to novel motion patterns during deployment.

Fixed Trajectory Dynamics. The predictor assumes that the leader trajectory follows continuous, physically plausible dynamics. Human operators, however, can produce discontinuous commands—lifting the input device, pausing mid-motion, or making abrupt corrections. These discontinuities violate the smoothness assumptions embedded in the LSTM’s learned dynamics model, potentially causing prediction errors during such transient events.

Dependency on Pre-trained LSTM Encoder. The proposed framework critically depends on pre-training the LSTM encoder via behavioral cloning before RL fine-tuning. This is not merely an optimization choice but a structural requirement for training stability. Without pre-training, three fundamental problems arise:

1. **The Moving Target Problem.** If the LSTM encoder is trained jointly with the RL policy from scratch, its weights update continuously during policy optimization. This causes the latent representation $\mathbf{h}_k$ to shift even for identical physical states. The critic, which learns $Q(\mathbf{h}_k, \boldsymbol{\tau}_k)$, faces a non-stationary input distribution—the definition of “state” changes under its feet. Consequently, the critic cannot converge, producing high-variance gradients that destabilize the actor and lead to policy collapse.
2. **The Chicken-and-Egg Problem.** Effective LSTM training requires coherent trajectory data that exhibits learnable temporal patterns. However, an untrained actor produces random, chaotic motions that contain no meaningful structure. The LSTM cannot learn delay dynamics from such noise, and without useful latent representations, the actor cannot improve. This circular dependency traps the system in a degenerate equilibrium where neither component learns.

3. **The Noisy Gradient Problem.** Supervised learning provides dense, informative gradients: “the predicted state was $\hat{s}$, the true state was s , adjust accordingly.” Reinforcement learning provides only a scalar reward signal that is sparse, delayed, and noisy. Training a complex sequence model like an LSTM purely from scalar rewards is extremely sample-inefficient. Pre-training with supervised learning solves the difficult representation learning problem with rich gradients, leaving RL to solve only the control problem.

Table 6.2 summarizes the consequences of omitting the pre-training phase.

Table 6.2 Impact of LSTM pre-training on training stability and convergence.

Property	With Pre-training	Without Pre-training
Critic input stability	Stable features	Non-stationary (shifting h_k)
Early exploration data	Useful	Uninformative noise
Gradient quality	Dense (BC) + Sparse (RL)	Sparse only
Convergence	Fast ($\sim 200k$ steps)	Very slow or divergent
Failure mode	Minor oscillations	Policy collapse

This dependency on pre-training implies that the framework cannot be deployed in a purely online learning setting. New deployment scenarios require collecting expert demonstrations first, which may not always be feasible. Developing methods that stabilize joint encoder-policy learning without supervised pre-training remains an open challenge.

6.3 Future Directions

Building upon our advanced delay-adaptive architecture—which successfully integrates explicit delay modeling with physics-aware state estimation—future research can extend this foundation toward more capable and deployable teleoperation systems. While the proposed framework demonstrates robust performance under stochastic delays, several promising extensions warrant further investigation:

- **Bilateral Teleoperation with Force Feedback:** This work addresses unilateral position tracking. Extending to bilateral teleoperation introduces additional challenges: the controller must simultaneously track leader motion and render contact forces to the operator. Under stochastic delays, maintaining passivity becomes critical to ensure energy dissipation and system stability. Integrating the proposed delay-adaptive encoder with passivity-based control frameworks presents a natural extension.
- **Online Adaptation:** The current policy is trained offline and cannot adapt to conditions outside its training distribution. In real-world deployment, operators may suddenly change their movement patterns, switch tasks, or encounter unexpected disruptions that require different control strategies. Developing test-time adaptation mechanisms that update encoder or policy parameters online—without catastrophic forgetting of baseline performance—would enable the system to handle novel trajectories and evolving operator behaviors that were not anticipated during training.
- **Integration with Visual Feedback:** The current framework operates on joint-space measurements. Many teleoperation scenarios involve visual feedback where the operator relies on camera images rather than proprioceptive signals. Extending the delay-adaptive encoder to handle image observations—potentially with different delay characteristics than proprioceptive data—would enable vision-based teleoperation under stochastic latency.

6.4 Conclusion

This thesis presented an end-to-end reinforcement learning framework for robust teleoperation under stochastic network delays. The proposed architecture addresses three fundamental limitations of existing approaches:

1. **Delay-Adaptive Prediction:** Unlike prior methods that assume fixed or bounded delays, our LSTM encoder explicitly receives the delay magnitude $\bar{d}_s^{(i)}$ as an input feature, enabling dynamic adaptation of the prediction horizon to instantaneous network conditions.
2. **Continuous State Estimation:** The autoregressive rollout mechanism maintains physically consistent state estimates between packet arrivals, eliminating the discontinuous jumps that destabilize conventional observer-based approaches.
3. **Direct Torque Control:** By outputting joint torques directly rather than feeding estimates to a cascaded PD controller, the architecture avoids error accumulation and enables the policy to learn nonlinear compensation strategies.

Experimental evaluation demonstrates that these design choices yield substantial improvements over state-of-the-art baselines:

- Under high-variance delay conditions (90–290 ms), the proposed method achieves mean tracking error 0.02 m, 30% reduce compares with the SOTA PDMC model.
- The method exhibits strong generalization to unseen trajectories, achieving mean position error of $\mu = 0.025$ m on randomized test trajectories where baseline performance degrades significantly by 40%.

These results validate the central hypothesis of this work: treating delay as an explicit, observable state feature—rather than unmodeled disturbance—is fundamental to achieving robust teleoperation over unreliable networks. Furthermore, enable RL directly to learn compensation policy can improve the system performance. The framework establishes a new paradigm for delay-aware robotic control and provides a foundation for deploying teleoperation systems in real-world scenarios with unpredictable network conditions.

A Appendix

A.1 Notation

General Conventions

- Lowercase bold letters denote vectors: $\mathbf{q}, \mathbf{x}, \boldsymbol{\tau}$
- Uppercase bold letters denote matrices: $\mathbf{M}, \mathbf{C}, \mathbf{J}$
- Superscript k denotes discrete time step: $\mathbf{q}^k$
- Subscript l denotes leader robot: $\mathbf{q}_l$
- Subscript f denotes follower robot: $\mathbf{q}_f$
- Overbar denotes normalized quantities: $\bar{\mathbf{q}}, \bar{d}_s$

System Variables

Symbol	Dimension	Description
n	$\mathbb{Z}_{>0}$	Number of robot joints (degrees of freedom)
$\mathbf{q}_l^k, \mathbf{q}_f^k$	$\mathbb{R}^n$	Joint positions of leader and follower
$\dot{\mathbf{q}}_l^k, \dot{\mathbf{q}}_f^k$	$\mathbb{R}^n$	Joint velocities of leader and follower
$\mathbf{s}_l^k, \mathbf{s}_f^k$	$\mathbb{R}^{2n}$	State vectors $[\mathbf{q}^\top, \dot{\mathbf{q}}^\top]^\top$
$\mathbf{x}_l^k, \mathbf{x}_f^k$	$\mathbb{R}^m$	End-effector positions of leader and follower
$\mathbf{e}^k$	$\mathbb{R}^n$	Joint tracking error $(\mathbf{q}_l^k - \mathbf{q}_f^k)$
$\boldsymbol{\tau}^k$	$\mathbb{R}^n$	Joint torque command
$\tilde{\boldsymbol{\tau}}^k$	$[-1, 1]^n$	Normalized torque (network output)
$\boldsymbol{\tau}_{\max}$	$\mathbb{R}^n$	Maximum torque limits

Delay Variables

Symbol	Dimension	Description
d_s^k	$\mathbb{R}_{\geq 0}$	Stochastic network delay (seconds)
$\bar{d}_s^k$	$[0, 1]$	Normalized delay magnitude
d_o	$\mathbb{R}_{\geq 0}$	Observation delay (constant)
$d_{\min}, d_{\max}$	$\mathbb{R}_{\geq 0}$	Delay range bounds
d_{norm}	$\mathbb{R}_{>0}$	Delay normalization factor
Δt	$\mathbb{R}_{>0}$	Control time step duration

Observation and History

Symbol	Dimension	Description
$\mathbf{o}_k$	$\mathbb{R}^{n_o}$	Observation vector at step k
$\mathbf{Z}_k$	$\mathbb{R}^{T \times 36}$	History sequence of T observation frames
$\mathbf{z}^{(i)}$	$\mathbb{R}^{36}$	Single observation frame
$\mathcal{H}_\tau^k$	$\mathbb{R}^{M \times n}$	Action history of past M actions
T	$\mathbb{Z}_{>0}$	History sequence length
M	$\mathbb{Z}_{>0}$	Action history length

Network Architecture

Symbol	Dimension	Description
$\mathbf{h}_k$	$\mathbb{R}^{d_h}$	LSTM hidden state / latent representation
$\mathbf{c}_k$	$\mathbb{R}^{d_h}$	LSTM cell state
d_h	$\mathbb{Z}_{>0}$	Hidden state dimension
$\hat{\mathbf{s}}_l^k$	$\mathbb{R}^{2n}$	Predicted leader state
H_{ar}	$\mathbb{Z}_{\geq 0}$	Autoregressive rollout steps
$\boldsymbol{\mu}_k, \boldsymbol{\sigma}_k$	$\mathbb{R}^n$	Policy mean and standard deviation
$Q_\psi(\cdot)$	$\mathbb{R}$	Critic Q-function

Training Variables

Symbol	Dimension	Description
$\mathcal{L}_{BC}$	$\mathbb{R}$	Behavioral cloning loss
$\mathcal{L}_{pred}$	$\mathbb{R}$	Prediction loss
$\mathcal{L}_{SAC}$	$\mathbb{R}$	SAC actor loss
λ_{aux}	$\mathbb{R}_{>0}$	Auxiliary loss weight
γ	$[0, 1)$	Discount factor
α	$\mathbb{R}_{>0}$	SAC temperature parameter
$\bar{\mathcal{H}}$	$\mathbb{R}$	Target entropy

Reward Function

Symbol	Dimension	Description
r_k	$\mathbb{R}$	Total reward at step k
r_k^{pos}, r_k^{vel}	$\mathbb{R}$	Position and velocity tracking rewards
$r_k^{effort}, r_k^{smooth}$	$\mathbb{R}$	Effort and smoothness penalties
w_p, w_v, w_u, w_s	$\mathbb{R}_{>0}$	Reward component weights
α_p, α_v	$\mathbb{R}_{>0}$	Exponential reward scaling factors

Mathematical Operators

Symbol	Description
$\ \cdot\ _2$	Euclidean (L2) norm
$\odot$	Element-wise (Hadamard) product
$\sigma(\cdot)$	Sigmoid function
$\tanh(\cdot)$	Hyperbolic tangent function
$\mathbb{E}[\cdot]$	Expectation operator
∇_{θ}	Gradient with respect to θ
$\lceil \cdot \rceil$	Ceiling function
$\mathbf{1}[\cdot]$	Indicator function

Bibliography

- [1] P. J. Choi, R. J. Oskouian, and R. S. Tubbs, “Telesurgery: Past, Present, and Future,” *Cureus*, vol. 10, no. 5, 2018.
- [2] V. K. Manupati, M. G. Krishnan, M. L. R. Varela, and J. Machado, “Telefacturing based distributed manufacturing environment for optimal manufacturing service by enhancing the interoperability in the hubs,” *Journal of Engineering*, vol. 2017, 2017.
- [3] C. Ruoff, *Teleoperation and Robotics in Space (Ingenieria Mecanica y Maquinaria)*. American Institute of Aeronautics & Astronautics, 1994.
- [4] K. Nagatani, S. Kiribayashi, Y. Okada, K. Otake, K. Yoshida, S. Tadokoro *et al.*, “Emergency response to the Fukushima Daiichi Nuclear Power Plant accident: Unmanned ground vehicle based surveillance missions,” *Journal of Field Robotics*, vol. 30, no. 1, pp. 44–63, 2013.
- [5] P. J. Hokayem and M. W. Spong, “Bilateral teleoperation: An historical survey,” *Automatica*, vol. 42, no. 12, pp. 2035–2057, 2006.
- [6] D. P. Losey, C. G. McDonald, S. T. Battaglia, and M. K. O’Malley, “A review of intent detection, arbitration, and communication aspects of shared control anthropomorphic robots,” *Applied Bionics and Biomechanics*, vol. 2018, 2018.
- [7] L. McCutcheon and S. Fallah, “Adaptive PD Control Using Deep Reinforcement Learning for Local-Remote Teleoperation with Stochastic Time Delays,” in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 7046–7053.
- [8] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [9] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 1861–1870.
- [10] V. R. Konda and J. N. Tsitsiklis, “Actor-critic algorithms,” in *Advances in Neural Information Processing Systems (NIPS)*, vol. 12, 1999, pp. 1008–1014.
- [11] R. J. Anderson and M. W. Spong, “Bilateral Control of Teleoperators with Time Delay,” *IEEE Transactions on Automatic Control*, vol. 34, no. 5, pp. 494–501, 1989.
- [12] G. Niemeyer and J. J. Slotine, “Stable Adaptive Teleoperation,” *IEEE Journal of Oceanic Engineering*, vol. 16, no. 1, pp. 152–162, 1991.
- [13] R. Lozano, N. Chopra, and M. W. Spong, “Passivity analysis of bilateral teleoperation systems with time varying delay,” *Automatica*, vol. 38, no. 9, pp. 1549–1559, 2002.
- [14] D. A. Lawrence, “Stability and transparency in bilateral teleoperation,” *IEEE Transactions on Robotics and Automation*, vol. 9, no. 5, pp. 624–637, 1993.
- [15] O. J. M. Smith, “Closed Control of Loops with Dead Time,” *Chemical Engineering Progress*, vol. 53, no. 5, pp. 217–219, 1957.

- [16] S. Sirouspour and A. Shahdi, “Model predictive control for transparent bilateral teleoperation under time delay,” *IEEE Transactions on Robotics*, vol. 22, no. 6, pp. 1131–1144, 2006.
- [17] Y. Zheng, M. J. Brudnak, P. Jayakumar, J. L. Stein, and T. Ersal, “A predictor-based framework for delay compensation in networked closed-loop systems,” in *ASME 2018 Dynamic Systems and Control Conference*, 2018.
- [18] —, “A delay compensation framework for predicting heading in teleoperated ground vehicles,” *IEEE/ASME Transactions on Mechatronics*, vol. 24, no. 6, pp. 2365–2376, 2019.
- [19] —, “An experimental evaluation of a model-free predictor for delay compensation in teleoperated ground vehicles,” *IEEE Transactions on Human-Machine Systems*, vol. 50, no. 6, pp. 535–544, 2020.
- [20] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, “Planning and acting in partially observable stochastic domains,” *Artificial Intelligence*, vol. 101, no. 1-2, pp. 99–134, May 1998. [Online]. Available: [https://doi.org/10.1016/S0004-3702\(98\)00023-X](https://doi.org/10.1016/S0004-3702(98)00023-X)
- [21] E. Altman and P. Nain, “Closed-loop control with delayed information,” *ACM SIGMETRICS Performance Evaluation Review*, vol. 20, no. 1, pp. 193–204, jun 1992.
- [22] T. J. Walsh, A. Nouri, L. Li, and M. L. Littman, “Planning and learning in environments with delayed feedback,” in *Machine Learning: ECML 2007*, ser. Lecture Notes in Computer Science, vol. 4701. Springer, 2007, pp. 442–453. [Online]. Available: https://doi.org/10.1007/978-3-540-74958-5_44
- [23] S. Nath, M. Baranwal, and H. Khadilkar, “Revisiting State Augmentation Methods for Reinforcement Learning with Stochastic Delays,” in *Proceedings of the Association for Computing Machinery*, 2021, pp. 1346–1355.
- [24] Q. Wu, S. S. Zhan, Y. Wang, Y. Wang, C.-W. Lin, C. Lv, Q. Zhu, J. Schmidhuber, and C. Huang, “Boosting reinforcement learning with strongly delayed feedback through auxiliary short delays,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 53 973–53 998. [Online]. Available: <https://proceedings.mlr.press/v235/wu24bn.html>
- [25] J. Saito, K. Narisawa, and A. Shinohara, “Prediction for control delay on reinforcement learning,” in *Proceedings of the 4th International Conference on Agents and Artificial Intelligence - Volume 1: SSML*, 2012, pp. 579–586.
- [26] V. Firoiu, T. Ju, and J. B. Tenenbaum, “At human speed: Deep reinforcement learning with action delay,” in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 1514–1523.
- [27] B. Chen, M. Xu, L. Li, and D. Zhao, “Delay-Aware Model-Based Reinforcement Learning for Continuous Control,” *Neurocomputing*, vol. 450, pp. 119–128, 2021.
- [28] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, “Mastering diverse domains through world models,” *arXiv preprint arXiv:2301.04104*, 2023.
- [29] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, and S. Levine, “Learning complex dexterous manipulation with deep reinforcement learning and demonstrations,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2017.
- [30] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011, pp. 627–635.

- [31] E. Derman, G. Dalal, and S. Mannor, “Acting in delayed environments with non-stationary markov policies,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [32] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [33] A. Gu, K. Goel, and C. Ré, “Efficiently modeling long sequences with structured state spaces,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [34] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023.